

TRABAJO DE FIN DE GRADO

Grado en Ingeniería de Computadores

ESCUELA POLITÉCNICA SUPERIOR

Estimación de intenciones de cruce mediante modelos
basados en grafos y datos de cámaras de tráfico globales

Rubén Sánchez Librero

Tutor: Augusto Luis Ballardini

Cotutora: Angie Nataly Melo Castillo

2026

UNIVERSIDAD DE ALCALÁ

ESCUELA POLITÉCNICA SUPERIOR

Grado en Ingeniería de Computadores

Trabajo de Fin de Grado

**Estimación de intenciones de cruce mediante modelos basados
en grafos y datos de cámaras de tráfico globales**

Autor: Rubén Sánchez Librero

Tutor: Augusto Luis Ballardini

Cotutora: Angie Nataly Melo Castillo

Tribunal:

Presidente: Javier Pedro Carracedo

Vocal: Rubén Izquierdo Gonzalo

Suplente: Antonio Guerrero Baquero

Fecha de depósito: 07 de Julio de 2026

Agradecimientos

Este trabajo está dedicado a todas las personas que han confiado en mí.

Gracias a mi familia, que sin su respaldo diario, la incertidumbre del proyecto hubiera sido mucho más densa. En especial, a mi tío Alberto, que ha sido la persona que siempre me ha cogido el teléfono cuando más lo necesitaba.

Gracias a mis compañeros y amigos del grado. Sin ellos las horas de clase hubieran sido muy diferentes, sin olvidarme de las risas y momentos compartidos.

Gracias a mis amigos de Guada y al centro juvenil de Salesianos, por haberme dado el enfoque real y humanitario para todo lo que haga.

Gracias a todas las personas que han estado relacionadas en el proyecto, de una manera más profesional. Un gran alarde a mi tutor, Augusto, por la apuesta en mí para el desarrollo del proyecto y sus recomendaciones para el acabado del mismo. A su vez, a mi compañero Adrián, por haberme facilitado y acercado mi proyecto desde el suyo.

Resumen

El objetivo del trabajo es la construcción de un sistema completo de predicción de la intención de cruce de peatones a partir de cámaras de tráfico públicas. Un *pipeline* automatizado captura, rastrea y etiqueta secuencias desde diferentes cámaras públicas distribuidas por todo el mundo, generando un dataset específico pero con capacidad de ser ampliado y adaptado a nuevas cámaras. Además, se ha desarrollado un ejemplo de red neuronal de grafos espacio-temporal, con posibilidad de un modo multipeatón. Por último, se han evaluado y comparado diferentes estilos de grafo, siendo el estilo *feature-nodes* el que mejores resultados ofrece frente al estilo *interaction*.

Palabras clave: intención de cruce, redes neuronales de grafos, cámaras fijas, dataset global, seguridad vial.

Abstract

The aim of this work is the construction of a complete pedestrian crossing intention prediction system based on public traffic cameras. An automated *pipeline* captures, tracks and labels sequences from different public cameras distributed worldwide, generating a specific dataset with the capacity to be extended and adapted to new cameras. In addition, a spatio-temporal graph neural network has been developed, supporting a multi-pedestrian mode. Finally, different graph styles have been evaluated and compared, with the *feature-nodes* style yielding the best results over the *interaction* style.

Keywords: crossing intention, graph neural networks, fixed cameras, global dataset, road safety.

Índice

Agradecimientos	iii
Resumen	v
Abstract	vii
Índice	viii
Índice de figuras	xi
Índice de tablas	xiv
Lista de acrónimos	xv
Lista de símbolos	xv
1 Introducción	1
1.1 Motivación	1
1.2 Contexto y posicionamiento	2
1.3 Objetivos	3
1.4 Organización de la memoria	3
2 Estado del arte	5
2.1 JAAD	5
2.2 <i>Features</i> relevantes para la predicción de intención de cruce	6
2.2.1 Señales individuales del peatón	6
2.2.2 Señales de posición y geometría	6
2.2.3 Señales del entorno	7
2.2.4 Señales sociales y de grupo	7
2.3 Enfoques de modelado	7
2.3.1 Modelos secuenciales	7
2.3.2 Redes de atención y Transformers	7
2.3.3 Redes neuronales de grafos	7
2.4 Síntesis y hueco que cubre este trabajo	8
3 Marco teórico	9
3.1 Aprendizaje profundo	9
3.2 Aprendizaje en secuencias	10
3.3 Grafos y su representación	11
3.4 Redes neuronales de grafos y <i>message passing</i>	12
3.5 GraphConv: convolución de Weisfeiler-Leman generalizada	13

3.6	Grafos espacio-temporales	14
4	Herramientas y tecnologías	17
4.1	Detección de objetos: YOLO26 (Ultralytics)	17
4.2	Seguimiento de múltiples objetos: ByteTrack	18
4.3	Procesamiento de imagen e interfaz visual: OpenCV	19
4.4	Adquisición de <i>streams</i> de vídeo: Streamlink y FFmpeg	19
4.5	Ecosistema PyTorch	19
4.5.1	PyTorch	19
4.5.2	PyTorch Geometric	19
4.5.3	PyTorch Lightning	20
4.5.4	TorchMetrics y scikit-learn	20
4.6	Contenerización: Docker	20
4.7	Control de versiones: Git y GitHub	21
5	Diseño del sistema: captura, etiquetado y dataset	25
5.1	Visión general del <i>pipeline</i>	25
5.2	Paso 1: Captura de secuencias (<code>datasetMaker.py</code>)	26
5.2.1	Configuración de zonas de interés (ROI)	26
5.2.2	Modos de grabación	27
5.2.3	Filtro de personas en vehículos	28
5.2.4	Grabación sin <i>heads-up display</i>	28
5.3	Orquestador multicámara (<code>recopilar.py</code>)	28
5.4	Paso 2: Etiquetado automático (<code>etiquetador.py</code>)	29
5.4.1	Esquema del CSV (22 columnas)	29
5.4.2	Decisiones de diseño relevantes	31
5.5	Dataset resultante	31
5.5.1	Cámaras y cobertura geográfica	31
5.5.2	Estadísticas del CSV bruto y cribado	33
5.5.3	Split de entrenamiento y prueba	33
6	Modelo GNN y metodología de entrenamiento	35
6.1	Construcción de grafos (<code>DatasetAmpliado</code>)	35
6.1.1	Estilo <i>interaction</i>	35
6.1.2	Estilo <i>feature-nodes</i>	36
6.2	Arquitectura del modelo	37
6.2.1	<code>GNNModel</code> — núcleo convolucional	38
6.2.2	<code>GraphGNNModel</code> — cabeza de clasificación	38
6.2.3	<code>GraphLevelGNN</code> — módulo <code>Lightning</code>	38
6.3	Tratamiento del desbalance de clases (<code>pos_weight</code>)	38
6.4	Métricas de evaluación y monitorización	38
6.4.1	AUC-ROC (métrica principal)	38
6.4.2	Recall por clase	39
6.4.3	Anticipación por horizonte (<code>cross_horizon</code>)	39
6.5	Configuración del entrenamiento	39
7	Pruebas y resultados	41
7.1	Ejecución 1: modo colapso sin <code>pos_weight</code>	41

7.2	Ejecución 2: EarlyStopping sobre val_f1	41
7.3	Ejecución 3: configuración de referencia (interaction)	42
7.4	Ejecución 4: experimento con pos_weight = 0,1	42
7.5	Ejecución 5: estilo feature-nodes — mejor resultado	42
7.6	Tabla comparativa de todas las ejecuciones	43
7.7	Análisis de la anticipación por horizonte (ejecución 5)	44
7.8	Discusión de la mejora del estilo feature-nodes	45
8	Discusión y limitaciones	47
8.1	Interpretación de los resultados	47
8.2	Comparación con el enfoque sobre JAAD (BAL.003)	48
8.3	Limitaciones	48
8.3.1	Dependencia de la calidad del <i>tracker</i>	48
8.3.2	Ventana temporal fija de tres fotogramas	49
8.3.3	Evaluación sobre solo dos cámaras de test	49
8.3.4	<i>Features</i> estimadas sin calibración métrica	49
8.3.5	Ausencia de inferencia en tiempo real	49
8.3.6	Balance de clases artificial mediante descarte	49
8.3.7	Sin validación cruzada	49
9	Conclusiones y trabajo futuro	51
9.1	Conclusiones	51
9.2	Trabajo futuro	52
9.2.1	Corto plazo	52
9.2.2	Medio plazo	52
9.2.3	Largo plazo	52
	Bibliografía	53

Índice de figuras

1.1	Intersección urbana con grafo de relaciones superpuesto.	2
2.1	Ejemplo de anotación del dataset JAAD: <i>bounding box</i> , etiqueta de acción e intención de cruce (adaptado de [2]).	6
2.2	Tipos de grafo empleados en predicción de trayectoria peatonal: (a) espacial multi- <i>feature</i> , (b) temporal, (c) espacio-temporal cruzado, (d) de escena (adaptado de [6]).	8
3.1	Red neuronal <i>feedforward</i> : las <i>features</i> del peatón se transforman capa a capa hasta producir la predicción de cruce.	10
3.2	RNN frente a LSTM desenrolladas en el tiempo: la LSTM añade un <i>cell state</i> $\mathbf{c}_t$ que preserva información a largo plazo.	11
3.3	Convolución 2D frente a convolución sobre grafo (adaptado de [8]).	12
3.4	Red neuronal convolucional sobre grafos con dos capas Gconv y activaciones ReLU (adaptado de [8]).	13
3.5	Propagación de información a través de las capas ocultas de una GNN (adaptado de [8]).	14
3.6	Grafo espacio-temporal: cada v_t representa el estado de la escena en el instante t (adaptado de [10]).	15
4.1	Stack tecnológico del <i>pipeline</i> : herramientas organizadas por etapa de captura, etiquetado y entrenamiento, sobre una imagen Docker base común.	17
4.2	Detección de personas con YOLO26: cajas delimitadoras y etiquetas de clase sobre una escena real (fuente: [11]).	18
4.3	Arquitectura de Docker: cliente, <i>daemon</i> y registro (fuente: https://geekflare.com/es/docker-architecture/).	21
4.4	Página principal del repositorio en GitHub (https://github.com/invett/TFG.BAL.004.GLOBAL-CAM), con la estructura de directorios del proyecto.	21
4.5	Historial de <i>commits</i> del proyecto en GitHub, donde cada entrada corresponde a un cambio significativo en el código o la configuración del sistema.	22
4.6	Wiki del repositorio en GitHub con la documentación interna del proyecto: decisiones de diseño, resultados de experimentos y guía de instalación.	23

5.1	Flujo del <i>pipeline</i> : captura (Paso 1), etiquetado (Paso 2) y cribado + entrenamiento GNN (Paso 3).	26
5.2	Etapas de configuración de máscaras ROI (OpenCV) sobre la cámara 02 (Jackson Town Square, Wyoming): acera, calzada, semáforos, paso de cebra y vista global con todas las zonas activas.	27
5.3	Lógica del orquestador multicámara (<i>recopilar.py</i>): flujo principal y condiciones de <i>skip</i> / fin de sesión.	29
5.4	Estructura del CSV de <i>features</i> : 22 columnas organizadas en cuatro grupos (metadatos, compatibles JAAD, propias y etiqueta).	30
5.5	Fotogramas representativos de las diez cámaras del dataset, ordenados por ID. Las cámaras 07 y 11 (fondo naranja) corresponden al conjunto de prueba.	32
5.6	Balance de clases antes y después de <i>criba.py</i> : el porcentaje de pistas not-crossing sube del 22 % al 30 %.	33
6.1	Grafo espacio-temporal estilo <i>interaction</i> . Tres fotogramas consecutivos ($t - 1, t, t + 1$) con tres peatones por fotograma. Las aristas espaciales (verde discontinuo) conectan peatones del mismo fotograma; las aristas temporales (naranja) conectan el mismo peatón entre fotogramas consecutivos. Solo los nodos del fotograma central t emiten predicción.	36
6.2	Grafo espacio-temporal estilo <i>feature-nodes</i> . Cada peatón se descompone en 13 nodos: un nodo peatón central y 12 nodos <i>feature</i> (5 compatibles con JAAD en verde, 7 propios del sistema en morado) organizados en estructura de estrella. La arista social (naranja) conecta los nodos peatón de distintos peatones; la arista temporal (rojo) conecta el nodo peatón con su instancia en el fotograma siguiente.	37
6.3	Arquitectura del modelo GNN. El tensor de entrada ($N \times C_{IN}$) atraviesa dos capas GraphConv con $C_{HIDDEN} = 128$ unidades (GNNModel), se filtran los nodos del fotograma central t (GraphGNNModel) y una capa lineal con BCEWithLogitsLoss produce un logit por peatón (GraphLevelGNN).	37
6.4	Curvas de entrenamiento reales de JAAD_ampliado.py. Panel izquierdo: AUC-ROC de validación por época para el estilo <i>interaction</i> (azul, línea continua) y <i>feature-nodes</i> (rojo, línea discontinua). Panel derecho: pérdida de validación correspondiente. Los marcadores indican la época óptima seleccionada por ModelCheckpoint.	39
7.1	Matriz de confusión de la ejecución 5 sobre el conjunto de test. El color codifica el <i>recall</i> por clase real; los recuentos de FP y FN se derivan de los <i>recalls</i> reportados.	43
7.2	Comparativa de AUC-ROC (eje izquierdo) y anticipación >30 fotogramas (eje derecho) de las cinco ejecuciones. La ejecución 1 no produjo AUC-ROC válido por entrar en modo colapso.	44
7.3	Recall de crossing por <i>bucket</i> de horizonte temporal en la ejecución 5. El color va del verde (horizonte corto) al rojo (horizonte >1 segundo).	45

8.1	Compromiso entre el <i>recall</i> de not-crossing y la anticipación temprana. El tamaño de cada burbuja es proporcional al AUC-ROC de la ejecución correspondiente.	48
-----	---	----

Índice de tablas

5.1	Tipos de zona ROI configurables por cámara.	26
5.2	<i>Features</i> compatibles con JAAD.	30
5.3	<i>Features</i> propias del sistema.	30
5.4	Cámaras del dataset.	31
5.5	Estadísticas del CSV cribado.	33
6.1	Tipos de aristas en el grafo de estilo <i>interaction</i>	35
6.2	Configuración del módulo <i>Lightning</i>	38
6.3	<i>Buckets</i> de horizonte de anticipación (a 30 fps).	39
7.1	Resultados de la ejecución 2.	41
7.2	Resultados de la ejecución 3 (referencia).	42
7.3	Resultados de la ejecución 4 frente a la ejecución 3.	42
7.4	Resultados de la ejecución 5 (<i>feature-nodes</i>).	43
7.5	Comparativa de las cinco ejecuciones de entrenamiento.	43
7.6	Recall de <i>crossing</i> por <i>bucket</i> de horizonte (ejecución 5).	44

Lista de acrónimos

ADAS	Advanced Driver Assistance Systems.
CPU	Central Processing Unit.
CSV	Comma-Separated Values.
GCN	Graph Convolutional Network.
GNN	Graph Neural Network.
GPU	Graphics Processing Unit.
GRU	Gated Recurrent Unit.
HUD	Heads-Up Display.
JAAD	Joint Attention in Autonomous Driving.
LSTM	Long Short-Term Memory.
MOT	Multiple Object Tracking.
PIE	Pedestrian Intention Estimation.
PyG	PyTorch Geometric.
RNN	Recurrent Neural Network.
ROI	Region of Interest.
WL	Weisfeiler-Leman.
YOLO	You Only Look Once.

Capítulo 1

Introducción

En este capítulo se expone la motivación que justifica el problema existente, el contexto en el que se inscribe la contribución, los objetivos concretos que se persiguen y la organización del resto de contenidos de la memoria.

1.1 Motivación

Los peatones constituyen uno de los colectivos más vulnerables en el tráfico rodado. Según los datos de la Dirección General de Tráfico, en 2023 fallecieron en España más de 400 peatones en vías interurbanas y urbanas, lo que representa aproximadamente el 22 % del total de víctimas mortales. A escala global, la Organización Mundial de la Salud estima que 1,19 millones de personas mueren cada año en accidentes de tráfico, siendo los peatones responsables del 23 % de esas muertes [1].

Frente a estas cifras, los sistemas avanzados de asistencia a la conducción (Advanced Driver Assistance Systems (ADAS)) han evolucionado hasta incorporar módulos de percepción capaces de detectar peatones y estimar sus trayectorias futuras. Sin embargo, la detección física de un peatón no es suficiente para anticipar si va a cruzar la calzada en los próximos segundos. Para ello se pone el siguiente ejemplo: dos peatones en la misma posición y con la misma apariencia pueden tener intenciones completamente opuestas, mientras uno espera en el bordillo, el otro puede estar a punto de cruzar. La predicción de la intención de cruce va un paso más allá de la detección: busca responder la pregunta de si cruzará con suficiente antelación para que el sistema disponga de tiempo de reacción útil (véase Figura 1.1).

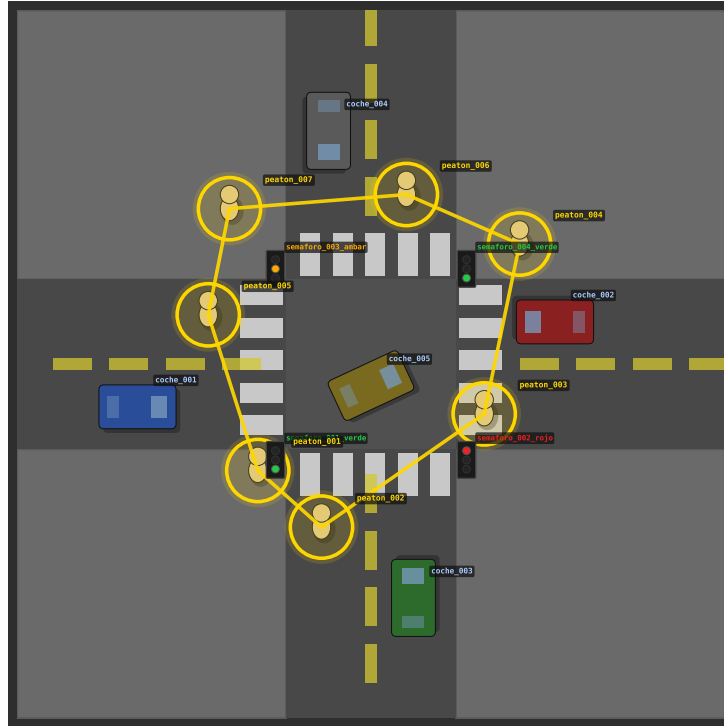


Figura 1.1: Intersección urbana con grafo de relaciones superpuesto.

La anticipación temprana es el criterio más exigente de este tipo de sistemas. No basta con detectar el cruce en el momento en que el peatón ya está sobre la calzada, podría ser una situación fatídica. Un sistema efectivo debe emitir la predicción correcta más de un segundo antes de que el peatón entre en "la zona de peligro".

Más allá del contexto de los vehículos autónomos, la predicción de intención de cruce tiene aplicaciones directas en infraestructuras inteligentes, como por ejemplo: semáforos adaptativos que prolongan la fase verde cuando un peatón está a punto de cruzar, sistemas de advertencia en pasos peligrosos, o cámaras de supervisión en intersecciones con mucho tráfico. Estas aplicaciones requieren procesar vídeo desde cámaras fijas instaladas en la propia infraestructura, no desde sensores embarcados en un vehículo, lo que motiva el enfoque basado en cámaras de tráfico públicas adoptado en este trabajo.

1.2 Contexto y posicionamiento

La mayor parte de los sistemas de predicción de intención de cruce existentes se entrenan sobre datasets capturados desde vehículos en movimiento, siendo Joint Attention in Autonomous Driving (JAAD) [2] y Pedestrian Intention Estimation (PIE) [3] los más utilizados. Ambos conjuntos de datos presentan una perspectiva propia de los vehículos equipados con una cámara frontal, lo cual hace que el punto de vista sea bajo. Esa perspectiva es muy adecuada para sistemas de conducción autónoma, pero no representa las condiciones de las cámaras fijas, que al estar elevadas, tienen un campo de visión más amplio y cubren siempre el mismo tramo de vía independientemente del tráfico.

Otro factor limitante de JAAD y PIE es su poca variedad geográfica: ambos datasets se grabaron en ciudades de Canadá, lo que restringe la diversidad de comportamiento peatonal, tipo de intersección y señalización. Un sistema entrenado exclusivamente sobre esos datos puede no generalizar bien a entornos con normas de circulación, infraestructura o densidad peatonal distintas.

El trabajo previo de esta misma línea de investigación, desarrollado bajo la dirección del mismo tutor (TFG BAL.003), abordó la predicción de intención de cruce empleando el dataset JAAD y una

arquitectura de red neuronal de grafos espacio-temporal. En su sección de trabajo futuro, ese trabajo identificó explícitamente la necesidad de ampliar el estudio a cámaras de tráfico públicas, como medio de reducir la dependencia de datasets capturados desde vehículo y de aumentar la diversidad geográfica.

El trabajo presente es directamente esa propuesta. Se han construido desde cero todas las herramientas necesarias para capturar, etiquetar y estructurar un dataset propio a partir de diez cámaras de tráfico públicas distribuidas en cinco continentes: Wyoming y New Hampshire (EEUU), Nueva Orleans (EEUU), Alberta (Canadá), San Petersburgo (Rusia), Sapporo y Tokio (Japón), Londres (Reino Unido) y Sídney (Australia). El dataset resultante contiene 478 secuencias con un total de 2,35 millones de filas etiquetadas y cubre condiciones lumínicas, culturales y de infraestructura muy variadas. Sobre este dataset se entrena y evalúa una Graph Neural Network (GNN) espacio-temporal con el mismo enfoque conceptual del trabajo precedente, lo que permite trazar una comparación implícita entre ambas aproximaciones.

La aportación central de este trabajo no es, por tanto, solo el modelo entrenado, sino el *pipeline* completo de obtención y estructuración de datos: un sistema automatizado que, dado un *stream* de vídeo público, produce de forma autónoma el dataset etiquetado listo para entrenar. Este *pipeline* es reproducible, escalable a nuevas cámaras sin intervención manual relevante, y está diseñado para ser compatible con el formato de etiquetado de JAAD, lo que facilita la comparación con trabajos previos.

1.3 Objetivos

Los objetivos del trabajo son los siguientes:

1. **Diseñar e implementar un *pipeline* automatizado de captura y etiquetado** de secuencias de vídeo de peatones a partir de *streams* de cámaras de tráfico públicas, incluyendo detección, seguimiento y extracción de *features* comportamentales y contextuales.
2. **Construir un dataset original de intención de cruce** en entorno de cámara fija, con cobertura geográfica diversa, etiquetado automático compatible con el formato JAAD, y un proceso de balanceo de clases documentado y reproducible.
3. **Entrenar y evaluar una red neuronal de grafos espacio-temporal** sobre el dataset construido, optimizando la anticipación temprana como criterio principal, y analizar el efecto de las decisiones de diseño (estilo de grafo, función de pérdida, métricas de monitorización) sobre los resultados finales.

1.4 Organización de la memoria

El formato de la memoria se organiza de la siguiente manera:

En primer lugar, el Capítulo 2 revisa el estado del arte en predicción de intención de cruce, con especial atención a las *features* empleadas, los datasets de referencia y los enfoques basados en redes neuronales de grafos. A continuación, el Capítulo 3 expone el marco teórico necesario: grafos, redes neuronales de grafos y la convolución GraphConv utilizada. Posteriormente, el Capítulo 4 describe las herramientas y tecnologías empleadas en el desarrollo. Más tarde, con el Capítulo 5 se detalla el diseño del sistema: el *pipeline* de captura, el etiquetador automático y las características del dataset resultante.

Más adelante, el Capítulo 6 describe el modelo GNN y la metodología de entrenamiento. A posteriori, el Capítulo 7 presenta los resultados obtenidos a lo largo del proceso de desarrollo, incluyendo el análisis comparativo entre configuraciones. Por otro lado, el Capítulo 8 discute los resultados y las limitaciones del trabajo. Terminando con el Capítulo 9 recogiendo las conclusiones y las líneas de trabajo futuro.

Capítulo 2

Estado del arte

La predicción de la intención de cruce de peatones es un problema de clasificación binaria cuya entrada es una secuencia de observaciones de un peatón sobre una intersección, y cuya salida es una estimación de la intención de cruce del peatón en próximos instantes. Este capítulo revisa el estado del arte en tres dimensiones: los datasets de referencia más actuales, las señales (*features*) que predicen con más exactitud la intención de cruce, y los enfoques de modelado más relevantes, con especial atención a los basados en redes neuronales de grafos.

2.1 JAAD

El dataset [JAAD](#) [2] es el conjunto de datos de referencia más utilizado para la tarea de predicción de intención de cruce. Contiene 346 clips de vídeo grabados desde una cámara montada en el parabrisas de un vehículo en movimiento, por ciudades de Canadá, grabando el comportamiento de los peatones (acción, mirada, gesto) y una etiqueta binaria de intención de cruce. La perspectiva del conductor hace que el peatón ocupe una fracción relativamente grande de la imagen, lo que facilita el análisis de la pose y la mirada.

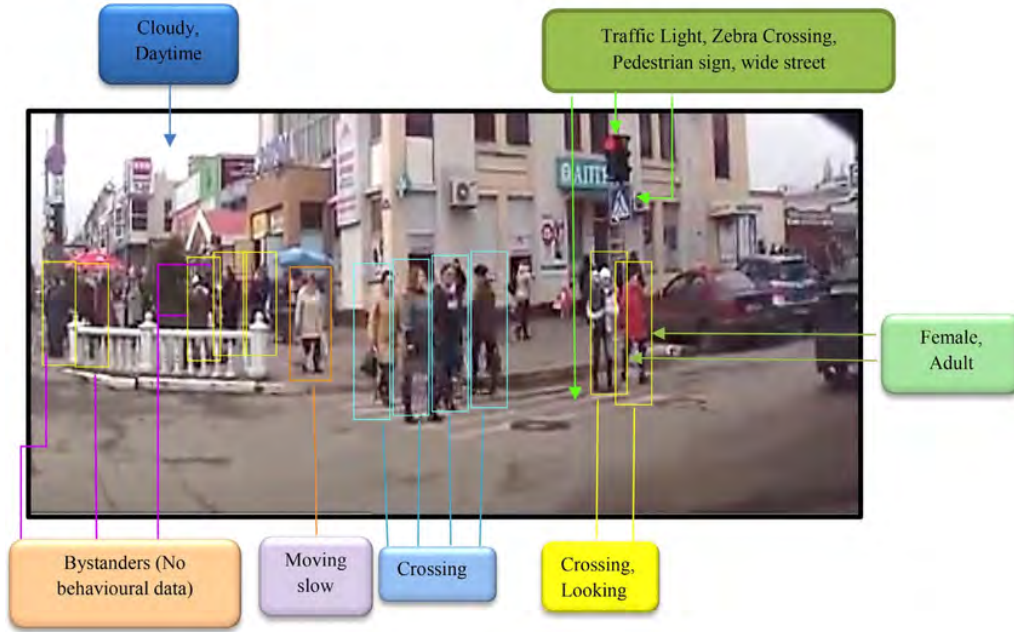


Figura 2.1: Ejemplo de anotación del dataset JAAD: *bounding box*, etiqueta de acción e intención de cruce (adaptado de [2]).

2.2 *Features* relevantes para la predicción de intención de cruce

Existen varias señales que contribuyen de forma consistente a la predicción de la intención de cruce [4]. Se agrupan en cuatro categorías.

2.2.1 Señales individuales del peatón

La **orientación y dirección de movimiento** (*heading*) son de las señales más informativas: un peatón que se desplaza encarado hacia la calzada tiene una trayectoria que apunta al cruce. En datasets con cámara frontal de vehículo, la orientación corporal se estima mediante modelos de pose (*keypoints*). En cámaras públicas elevadas, la pose es poco fiable para peatones distantes, por lo que se usa el ángulo derivado del desplazamiento del centroide entre fotogramas, más robusto a baja resolución [2].

La **acción instantánea** (*walking / standing*) separa la fase de espera de la fase de desplazamiento: un peatón parado en el bordillo indica que está a la espera, mientras que uno en movimiento hacia la calzada indica intención de cruzar. La acción se estima mediante un umbral sobre la velocidad de desplazamiento del centroide.

La **mirada** (*gaze*, *attention* en JAAD) se usa como señal directa de atención al tráfico. Sin embargo, esta señal requiere estimación de pose facial o de cabeza con suficiente resolución, lo que la hace poco fiable en cámaras públicas con peatones distantes. En este trabajo, la *feature attention* de JAAD se sustituye por *toward_road*, que proyecta el vector de movimiento sobre la máscara de calzada y resulta más robusta.

2.2.2 Señales de posición y geometría

La **distancia al paso de cebra** proporciona un gradiente continuo de proximidad al punto de cruce designado: un peatón que se acerca es más probable que cruce que uno que se aleja. La señal binaria *zebra_cross* (dentro / fuera del paso de cebra) indica si el peatón ha alcanzado ya la zona de cruce.

La **distancia al borde de la calzada** (`dist_road_edge`) cuantifica cuán cerca está el peatón de pisar la zona de riesgo, complementando la información del paso de cebra en situaciones en que el cruce no se produce por el paso de cebra.

2.2.3 Señales del entorno

El **estado del semáforo** es una señal causal directa: la mayoría de peatones regulan su decisión de cruce en función del color del semáforo. La variable `tl_color` (rojo / verde / ámbar) y el tiempo acumulado en ese estado (`tl_time_in_phase`) permiten al modelo capturar tanto el color actual como la proximidad al cambio de los mismos.

El **volumen de tráfico** (`n_vehicles_scene`) condiciona la percepción de riesgo del peatón: escenas con más vehículos suelen producir esperas más largas antes de cruzar.

2.2.4 Señales sociales y de grupo

Los humanos tienden a cruzar en grupo: la decisión de un peatón se ve influida por las acciones de los que le rodean. Las señales `proximity` (presencia de otro peatón a menos de una altura de cuerpo) y `n_peds_near` (número de peatones cercanos) capturan este efecto de imitación social, documentado en la literatura de comportamiento peatonal.

2.3 Enfoques de modelado

2.3.1 Modelos secuenciales

Los primeros trabajos en predicción de intención de cruce utilizaron modelos secuenciales como Long Short-Term Memory (LSTM) o Gated Recurrent Unit (GRU) para procesar trayectorias de *bounding boxes* a lo largo del tiempo. Estos modelos capturan la dinámica individual del peatón con horizontes de hasta 1–2 segundos, pero tratan a cada peatón de forma aislada, sin modelar la interacción con otros peatones ni con el entorno semántico de la escena [3].

2.3.2 Redes de atención y Transformers

Los modelos basados en mecanismos de atención (*Transformer*) abordan la predicción como una tarea de secuencia a secuencia, pesando la importancia relativa de cada fotograma pasado. Permiten capturar dependencias a largo plazo y, en versiones multi-cabeza, modelar relaciones entre peatones. Sin embargo, requieren secuencias de longitud fija y un número de parámetros significativamente mayor que los modelos basados en grafos, lo que puede ser problemático con datasets de tamaño medio.

2.3.3 Redes neuronales de grafos

Las redes neuronales de grafos ofrecen una representación natural para el problema de predicción de cruce: los peatones y los elementos de la escena son nodos, y sus relaciones (proximidad, conexión temporal) son aristas. Este enfoque permite propagar información entre peatones (contexto social) y entre el mismo peatón en momentos sucesivos (contexto temporal), usando minuciosamente la estructura que más informa la intención de cruce.

La convolución GraphConv de Morris et al. [5], basada en el test de Weisfeiler-Leman generalizado, ha mostrado buen comportamiento en grafos de densidad variable (como los que se producen al cambiar el número de peatones en escena) al normalizar la contribución de los vecinos por sus grados, estabilizando el entrenamiento sin necesidad de mecanismos de atención extras.

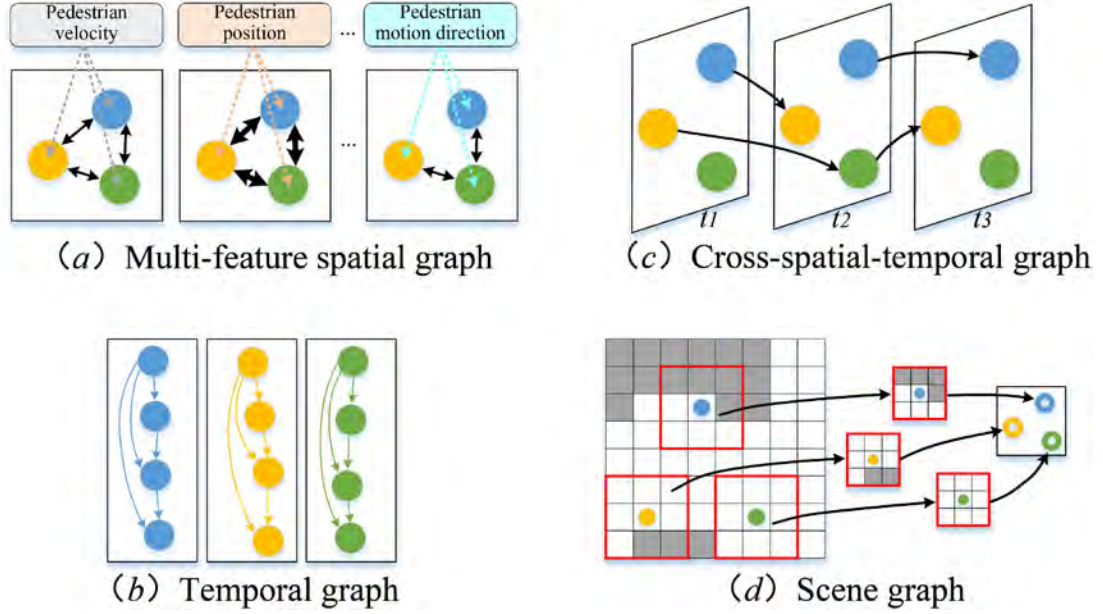


Figura 2.2: Tipos de grafo empleados en predicción de trayectoria peatonal: (a) espacial multi-*feature*, (b) temporal, (c) espacio-temporal cruzado, (d) de escena (adaptado de [6]).

2.4 Síntesis y hueco que cubre este trabajo

De la revisión anterior se extraen tres conclusiones. Primero, los datasets disponibles (JAAD, PIE) representan exclusivamente la perspectiva embarcada en vehículo y una única región geográfica. No existe, hasta donde llega la revisión, un dataset público de intención de cruce desde cámaras fijas y de geografía variadas. Segundo, las señales más informativas son la orientación y dirección de movimiento, la distancia al cruce, el estado del semáforo y el contexto social, todas ellas extraíbles de vídeo sin calibración métrica. Tercero, los modelos basados en GNN son la alternativa más adecuada para este tipo de problema porque modelan explícitamente las relaciones entre peatones, algo que los modelos secuenciales no hacen de forma nativa.

Este trabajo contribuye en dos frentes: un *pipeline* de obtención de datos que permite construir datasets similares a partir de cualquier cámara pública, y una evaluación del enfoque GNN espacio-temporal en condiciones de cámara fija y diversidad geográfica real.

Capítulo 3

Marco teórico

Este capítulo expone los conceptos teóricos que sustentan el modelo desarrollado. Se parte del aprendizaje profundo como marco general, se introduce el aprendizaje en secuencias y sus limitaciones para modelar interacciones entre peatones, y se presenta la representación en grafos como alternativa natural. A continuación, se describe el mecanismo de *message passing*, la convolución GraphConv empleada en este trabajo, y el grafo espacio-temporal que permite incorporar contexto temporal en la arquitectura.

3.1 Aprendizaje profundo

El aprendizaje profundo (*deep learning*) es una rama del aprendizaje automático basada en redes neuronales artificiales con múltiples capas de transformación. Una red neuronal *feedforward* recibe un vector de entrada, lo propaga a través de una serie de capas ocultas aplicando transformaciones lineales seguidas de funciones de activación no lineales (típicamente ReLU), y produce una salida. El entrenamiento minimiza una función de pérdida mediante descenso por gradiente con retropropagación (*backpropagation*), ajustando los pesos de la red iterativamente [7].

La profundidad de la red permite aprender representaciones jerárquicas: las capas inferiores capturan patrones locales y las capas superiores los combinan en conceptos de mayor nivel. En el contexto de este trabajo, la entrada de la red son los vectores de *features* de los peatones (orientación, distancia al cruce, estado del semáforo, etc.) y la salida es la probabilidad de cruce, como ilustra la Figura 3.1.

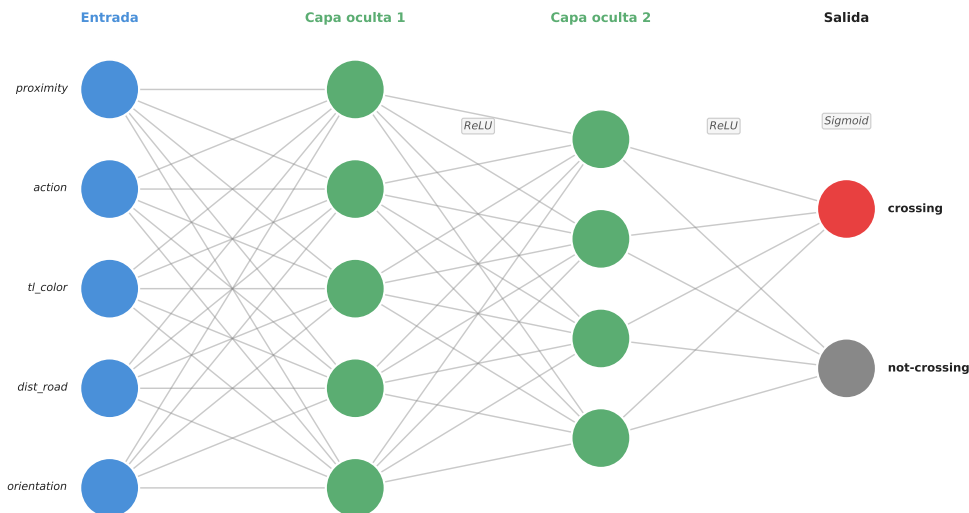


Figura 3.1: Red neuronal *feedforward*: las *features* del peatón se transforman capa a capa hasta producir la predicción de cruce.

Sin embargo, esta arquitectura asume que cada muestra de entrada es independiente de las demás. La trayectoria de un peatón viola esta suposición: el comportamiento observado en el fotograma t está condicionado por lo que ocurrió en $t-1$, $t-2$, etc. Ignorar esta dependencia temporal equivale a predecir la intención de cruce a partir de una instantánea aislada, descartando la información de movimiento acumulada en los fotogramas anteriores. Esto motiva el uso de modelos capaces de procesar datos en secuencia.

3.2 Aprendizaje en secuencias

La trayectoria de un peatón es una secuencia temporal: los *features* observados en el fotograma t dependen de los fotogramas anteriores. Las redes recurrentes (Recurrent Neural Network (RNN)) abordan este tipo de datos manteniendo un estado oculto $\mathbf{h}_t$ que se actualiza en cada paso: $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$. Sin embargo, sufren el problema del gradiente evanescente: la señal de error se atenúa al propagarse hacia atrás en secuencias largas, dificultando el aprendizaje de dependencias temporales lejanas.

Las redes **LSTM** resuelven este problema añadiendo un segundo estado, el *cell state* $\mathbf{c}_t$, que actúa como memoria a largo plazo y puede transportar información sin degradación a través de muchos pasos temporales (véase Figura 3.2). Esta arquitectura fue ampliamente adoptada en los primeros trabajos de predicción de intención de cruce para modelar la dinámica individual de cada peatón [3].

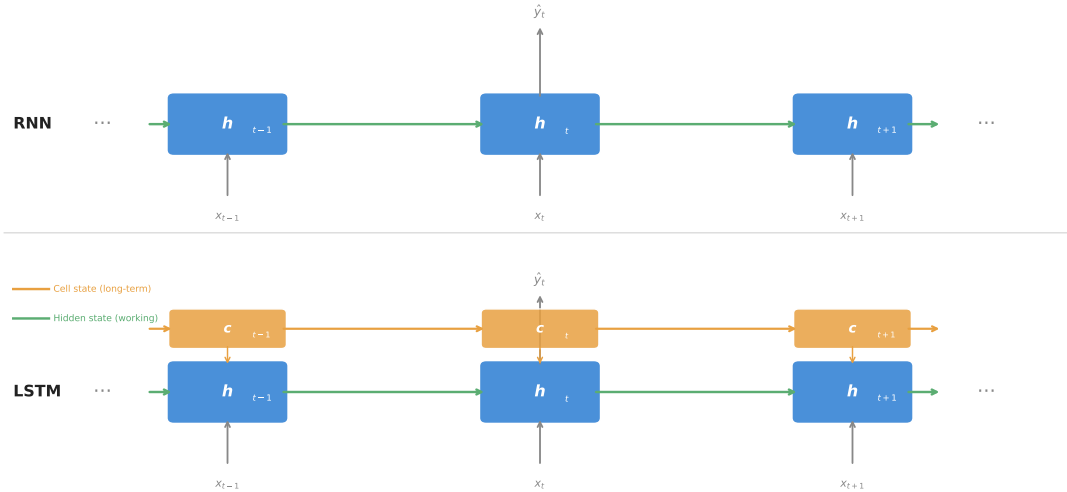


Figura 3.2: RNN frente a LSTM desenrolladas en el tiempo: la LSTM añade un *cell state* c_t que preserva información a largo plazo.

Sin embargo, tanto las RNN como las LSTM presentan una limitación estructural para este problema: procesan cada peatón de forma aislada, sin modelar las interacciones entre peatones ni con el entorno semántico de la escena. Como se argumentó en el capítulo 2, la decisión de cruce depende también del contexto social y del estado del semáforo, entre otros. Para ello, se necesita otra estructura de datos que permita modelar las relaciones entre entidades, de manera más cohesionada y general.

3.3 Grafos y su representación

Un grafo $G = (V, E)$ se define como un par formado por un conjunto de nodos V y un conjunto de aristas $E \subseteq V \times V$. Cada nodo $v \in V$ puede tener asociado un vector de atributos $\mathbf{x}_v \in \mathbb{R}^d$, y cada arista $(u, v) \in E$ puede llevar un peso o vector de atributos.

Los grafos son estructuras de datos irregulares: a diferencia de las imágenes (donde cada píxel tiene exactamente 8 vecinos) o de las secuencias (donde cada paso tiene exactamente uno anterior y uno posterior), el número de vecinos de un nodo puede variar de 0 a $|V| - 1$. Esta irregularidad es precisamente la que hace necesario un tipo especial de red neuronal.

En el contexto de este trabajo, los nodos representan peatones (o sus *features*, en el estilo *feature-nodes* descrito en el capítulo 6), y las aristas representan relaciones espaciales (peatones que comparten el mismo fotograma) o temporales (el mismo peatón en fotogramas consecutivos).

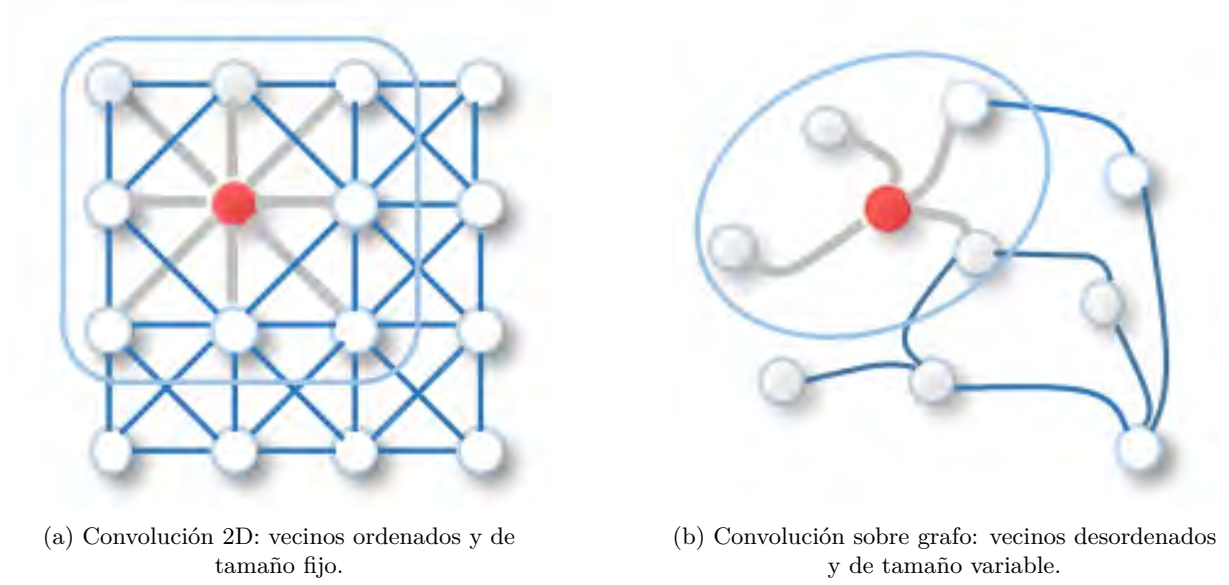


Figura 3.3: Convolución 2D frente a convolución sobre grafo (adaptado de [8]).

Las redes neuronales convencionales asumen entradas de tamaño fijo y orden definido, por lo que no pueden operar directamente sobre la topología irregular de un grafo donde el número de vecinos de cada nodo varía. Para aprender representaciones útiles a partir de esta estructura, es necesario un tipo de red diseñada específicamente para datos no euclidianos: las redes neuronales de grafos. Así no crearían problemas de irregularidades al no tener tamaño regular, pudiendo manejar de manera efectiva todo tipo de ocasiones.

3.4 Redes neuronales de grafos y *message passing*

Una red neuronal de grafos (GNN) aprende representaciones de nodos mediante la propagación iterativa de información a través de las aristas. La operación fundamental es el *message passing* [8]: en cada capa, cada nodo agrega los vectores de sus vecinos y actualiza su propio estado en función de esa agregación.

Formalmente, una capa de *message passing* calcula la representación del nodo v en la capa $\ell + 1$ como:

$$\mathbf{h}_v^{(\ell+1)} = \text{UPDATE}\left(\mathbf{h}_v^{(\ell)}, \text{AGG}\left(\left\{\mathbf{h}_u^{(\ell)} : u \in \mathcal{N}(v)\right\}\right)\right) \quad (3.1)$$

donde $\mathcal{N}(v)$ es el vecindario del nodo v , AGG es una función de agregación (suma, media, máximo) y UPDATE es una transformación lineal o no lineal. Después de L capas, cada nodo ha integrado información procedente de sus vecinos a distancia hasta L aristas.

La elección de la función de agregación determina la capacidad expresiva del modelo. Una suma simple da igual peso a todos los vecinos; una media normaliza por el grado; un máximo retiene solo la señal más fuerte. En el caso de los grafos peatón-escena empleados en este trabajo, donde el número de peatones en escena varía considerablemente entre fotogramas, la normalización por el grado es especialmente relevante para evitar que los nodos con muchos vecinos dominen la señal.

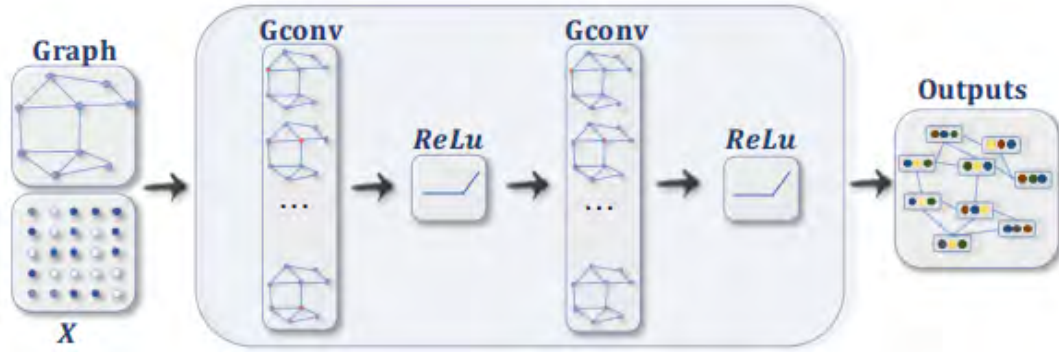


Figura 3.4: Red neuronal convolucional sobre grafos con dos capas Gconv y activaciones ReLU (adaptado de [8]).

El marco de *message passing* deja abierta la forma concreta de la función de agregación. Esta elección no es trivial: en las escenas grabadas con las cámaras de este trabajo, el número de peatones presentes varía entre uno y varios simultáneamente, lo que produce grafos con distribuciones de grado muy irregulares.

Es muy normal pensar que sin algún tipo de modificación extra para estas irregularidades daría problemas a largo plazo. Para este problema se utiliza la operación GraphConv, descrita en la siguiente sección, para abordar el problema de forma explícita.

3.5 GraphConv: convolución de Weisfeiler-Leman generalizada

La operación de convolución empleada en este trabajo es GraphConv, introducida por Morris et al. [5] como una generalización neuronal del test de isomorfismo de Weisfeiler-Leman (Weisfeiler-Leman (WL)). La fórmula de actualización es:

$$\mathbf{h}_v^{(\ell+1)} = \mathbf{W}_1 \cdot \mathbf{x}_v + \mathbf{W}_2 \cdot \sum_{u \in \mathcal{N}(v)} \frac{\mathbf{x}_u}{\sqrt{\deg(u) \cdot \deg(v)}} \quad (3.2)$$

donde $\mathbf{W}_1$ y $\mathbf{W}_2$ son matrices de pesos aprendibles, $\mathbf{x}_v$ es el vector de atributos del nodo v , y $\deg(v)$ es su grado (número de aristas incidentes). La normalización por $\sqrt{\deg(u) \cdot \deg(v)}$ garantiza que la contribución de cada vecino sea inversamente proporcional a la raíz cuadrada del producto de sus grados, lo que evita que los nodos con muchos vecinos amplifiquen desproporcionadamente la señal.

Esta normalización hace a GraphConv más estable que la Graph Convolutional Network (GCN) original de Kipf y Welling [9] en grafos donde la distribución de grados es irregular, como ocurre en escenas con densidad variable de peatones.

En el modelo empleado se apilan dos capas GraphConv, con una función de activación ReLU y *dropout* entre capas. Esto permite que cada nodo integre información de nodos a distancia 2 (vecinos de vecinos), lo que en el grafo espacio-temporal equivale a: influencia de otros peatones en el mismo fotograma, e influencia del mismo peatón dos pasos temporales atrás.

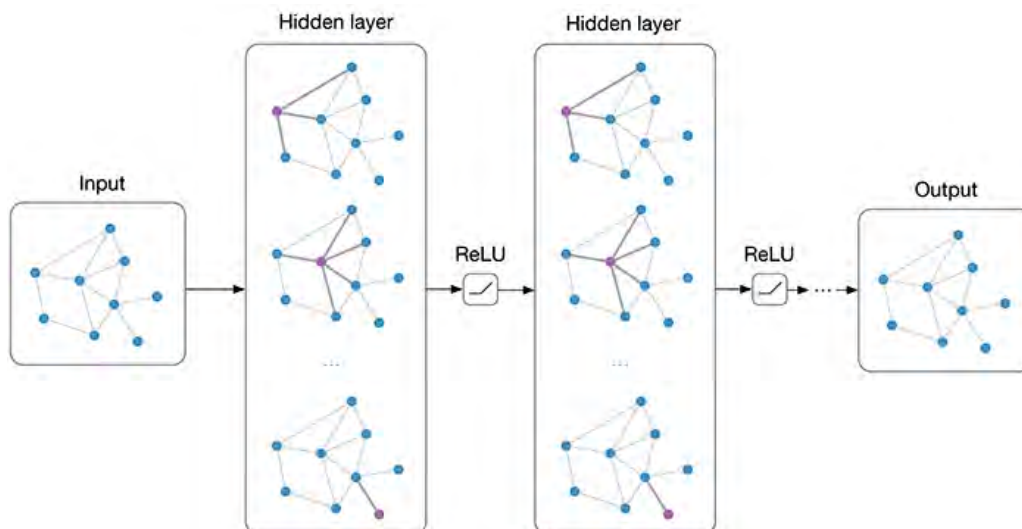


Figura 3.5: Propagación de información a través de las capas ocultas de una GNN (adaptado de [8]).

La convolución GraphConv captura relaciones entre nodos dentro de un mismo fotograma: qué peatones están cerca entre sí y cómo se influyen mutuamente. Sin embargo, predecir si un peatón va a cruzar requiere también entender cómo evoluciona su comportamiento a lo largo del tiempo: ¿está acelerando hacia la calzada? ¿Lleva varios fotogramas parado en el bordillo? Esta dimensión temporal no puede capturarse solamente con un grafo estático. Los grafos espacio-temporales pueden integrar la evolución desde los ámbitos espaciales y temporales de la escena en conjunto.

3.6 Grafos espacio-temporales

Un grafo espacio-temporal extiende la representación estática añadiendo la dimensión del tiempo: los nodos representan no solo entidades, sino entidades en un instante concreto, y las aristas pueden ser tanto espaciales (entre entidades en el mismo instante) como temporales (entre la misma entidad en instantes diferentes) [10].

En este trabajo se construye un grafo para cada triplete de fotogramas consecutivos $(t-1, t, t+1)$. Las **aristas espaciales** conectan todos los peatones presentes en el mismo fotograma entre sí, modelando la interacción social. Las **aristas temporales** conectan el mismo peatón (mismo identificador de ByteTrack) entre fotogramas consecutivos, modelando su dinámica individual.

La elección de una ventana de tres fotogramas responde a tres criterios. Primero, tres fotogramas son suficientes para capturar la dinámica de corto plazo más relevante para la intención de cruce (aceleración, deceleración, giro inminente); la investigación en comportamiento peatonal indica que la decisión de cruce se manifiesta principalmente en los instantes inmediatamente anteriores al cruce. Segundo, esta ventana evita las dependencias recurrentes de largo plazo (y sus problemas asociados de gradiente evanescente), manteniendo el entrenamiento estable. Tercero, cada grafo es una muestra independiente del dataset, lo que permite usar los mecanismos estándar de DataLoader de PyG sin necesidad de gestión de secuencias de longitud variable.

Este enfoque replica la metodología de ventanas espacio-temporales de la arquitectura de referencia JAAD [2], lo que facilita la comparación de resultados con trabajos previos sobre ese dataset.

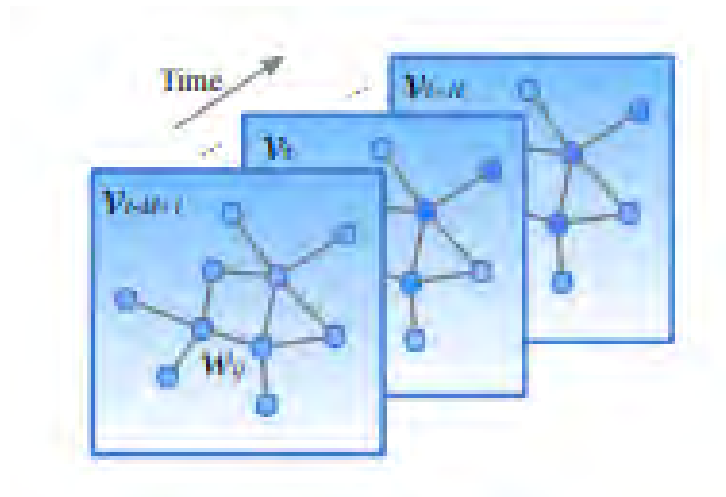


Figura 3.6: Grafo espacio-temporal: cada v_t representa el estado de la escena en el instante t (adaptado de [10]).

Los conceptos presentados en este capítulo —aprendizaje profundo, modelos secuenciales, representación en grafos, *message passing*, GraphConv y grafos espacio-temporales— constituyen la base teórica completa del modelo desarrollado en este trabajo. El capítulo siguiente describe las herramientas y bibliotecas concretas que hacen posible su implementación.

Capítulo 4

Herramientas y tecnologías

El desarrollo del sistema requirió un conjunto de herramientas heterogéneas que cubren desde la adquisición y procesamiento de vídeo hasta el entrenamiento y evaluación de modelos de aprendizaje profundo. Este capítulo describe las principales herramientas utilizadas, su papel en el *pipeline* y las versiones concretas empleadas. La Figura 4.1 muestra el conjunto completo organizado por etapa.



Figura 4.1: Stack tecnológico del *pipeline*: herramientas organizadas por etapa de captura, etiquetado y entrenamiento, sobre una imagen Docker base común.

4.1 Detección de objetos: YOLO26 (Ultralytics)

YOLO26 [11] es la versión 26 de la familia You Only Look Once (YOLO) (*You Only Look Once*), una arquitectura de detección de objetos en tiempo real de una sola pasada. A diferencia de los detectores en dos etapas (que primero proponen regiones de interés y luego las clasifican), YOLO predice directamente cajas delimitadoras y etiquetas de clase en una única inferencia sobre la imagen completa, lo que lo hace adecuado para su uso sobre *streams* de vídeo.

En el *pipeline* de este trabajo, YOLO26 se utiliza en los pasos 1 y 2 para detectar peatones, vehículos, semáforos y mascotas sobre cada fotograma. Se emplean dos pesos: el modelo *nano* (`yolo26n.pt`) para ejecución en Central Processing Unit (CPU) (menor consumo de memoria, suficiente para el modo de captura), y el modelo *large* (`yolo26l.pt`) para ejecución en Graphics Processing Unit (GPU) durante el etiquetado (mayor precisión, necesaria para generar *features* de calidad). Qué versión escoger depende de la capacidad de procesamiento, en el caso del proyecto se utilizó la versión *large* (`yolo26l.pt`) para un mayor rendimiento.

La detección de peatones es, por sí misma, un área de investigación muy activa. En los últimos años han proliferado variantes de la familia YOLO orientadas a mejorar la detección en condiciones difíciles —peatones de pequeña escala, escenas densas o tráfico complejo—, basadas en YOLOv5s [12], YOLOv8s [13], YOLOv9 [14] y YOLOv11 [15], así como modelos específicos para escenas densas [16], detección multiescala [17] y escenarios de tráfico complejos [18]. Una línea complementaria explora la fusión multispectral (RGB e infrarrojo) para mejorar la robustez en condiciones de baja iluminación [19], [20], [21]. En este trabajo se opta por YOLO26 por su disponibilidad e integración directa con el ecosistema Ultralytics, suficiente para la detección de peatones en cámaras fijas con buena iluminación.

El umbral de confianza mínima para aceptar una detección de peatón se fija en un valor conservador para reducir falsos negativos, sacrificando precisión en favor del *recall*: es preferible sobredetectar peatones y filtrarlos después por geometría que perder peatones que crucen sin ser detectados.



Figura 4.2: Detección de personas con YOLO26: cajas delimitadoras y etiquetas de clase sobre una escena real (fuente: [11]).

4.2 Seguimiento de múltiples objetos: ByteTrack

ByteTrack [22] es un algoritmo de seguimiento de múltiples objetos (Multiple Object Tracking (MOT)) que asigna identificadores estables a los objetos detectados a lo largo de los fotogramas de un vídeo. Su característica principal es que, a diferencia de los *trackers* que descartan las detecciones de baja confianza, ByteTrack las incluye en el proceso de asociación empleando una segunda ronda de emparejamiento con el filtro de Kalman, lo que reduce la tasa de pérdida de identidad en oclusiones y huecos de detección.

ByteTrack está integrado en la librería Ultralytics como el *tracker* por defecto. En el contexto de este trabajo, la estabilidad del identificador de ByteTrack es crítica: las *features* que dependen de la trayectoria (`action`, `orientation`, `toward_road`, `cross_horizon`), la etiqueta `cross` asignada

por trayectoria, y las aristas temporales del grafo espacio-temporal dependen todas de que el mismo peatón mantenga el mismo identificador entre fotogramas consecutivos, evitando falsos positivos y duplicaciones de una misma persona.

4.3 Procesamiento de imagen e interfaz visual: OpenCV

OpenCV (*Open Source Computer Vision Library*) es la librería estándar de facto para procesamiento de imagen y vídeo en Python. En este trabajo se emplea para tres funciones: la lectura y escritura de fotogramas de vídeo, la gestión de la interfaz interactiva de dibujo de máscaras Region of Interest (ROI) del generador de dataset, y el análisis de color por segmentación HSV para determinar el estado de los semáforos.

La interfaz de máscaras permite al usuario dibujar polígonos con el ratón sobre un fotograma congelado del *stream* para definir las zonas de acera, calzada, semáforos y paso de cebra. Estas máscaras se guardan en formato JSON y se reutilizan en todas las sesiones de captura y etiquetado posteriores de la misma cámara, de forma que la configuración inicial manual es un coste que se amortiza sobre todas las secuencias capturadas.

4.4 Adquisición de *streams* de vídeo: Streamlink y FFmpeg

Streamlink es una herramienta de línea de comandos que extrae la URL real del flujo de vídeo de plataformas de *streaming* (YouTube, EarthCam, etc.) a partir de la URL pública de la cámara. El flujo extraído se entrega directamente a OpenCV mediante un proceso de FFmpeg, que actúa como decodificador y recodificador de vídeo en segundo plano.

FFmpeg se usa además para comprimir los clips grabados con el *bitrate* configurado, reduciendo el tamaño en disco sin degradar los detalles visuales necesarios para el etiquetado posterior.

4.5 Ecosistema PyTorch

El núcleo de la fase de entrenamiento se construye sobre tres componentes del ecosistema de PyTorch que actúan en capas continuas: PyTorch proporciona la base tensorial y el motor de diferenciación automática; PyTorch Geometric añade las primitivas de grafos necesarias para el modelo GNN; y PyTorch Lightning estructura el bucle de entrenamiento y la gestión de *callbacks*.

4.5.1 PyTorch

PyTorch [7] es el *framework* de aprendizaje profundo sobre el que se construye toda la parte de modelado. Proporciona tensores multidimensionales con diferenciación automática (*autograd*), soporte para GPU mediante CUDA, y un ecosistema de librerías especializadas. En el proyecto se emplea la versión 2.5.1 con CUDA 12.1, con capacidad de poder ser alternativo tanto de modelo (por la versión de la GPU usada), como por el usado en los modelos *nano*.

4.5.2 PyTorch Geometric

PyTorch Geometric (PyG) [23] es la librería de referencia para el entrenamiento de redes neuronales de grafos sobre PyTorch. Implementa las operaciones de *message passing* de forma eficiente (dispersa) y proporciona la clase *Data* para representar grafos individuales y *Dataset* / *DataLoader* para gestionarlos en lotes. La convolución *GraphConv* de Morris et al. [5] está implementada en PyG como `torch_geometric.nn.GraphConv`, empleándose la versión 2.7.0.

La serialización de los grafos construidos se realiza mediante el mecanismo `InMemoryDataset` de PyG, que guarda los objetos `Data` en archivos `.pt` y los recarga sin reconstruirlos en ejecuciones posteriores, lo que reduce el tiempo de inicio del entrenamiento a partir de la segunda ejecución.

4.5.3 PyTorch Lightning

PyTorch Lightning [24] es una capa de abstracción sobre PyTorch que estandariza el bucle de entrenamiento, validación y test y gestiona automáticamente los aspectos de ingeniería (*checkpoints*, *logging*, *callbacks*, estrategias de paralelismo). En este trabajo se emplea para estructurar el módulo `GraphLevelGNN`, que encapsula el modelo, la función de pérdida, el optimizador y los *callbacks* de monitorización. Todo ello con la versión 2.6.1.

Los *callbacks* más relevantes son `ModelCheckpoint` y `EarlyStopping`, configurados para monitorizar el AUC-ROC en validación. La justificación de esta elección de métrica de monitorización se detalla en el capítulo 6.

4.5.4 TorchMetrics y scikit-learn

TorchMetrics (versión 1.9.0) proporciona implementaciones de métricas de clasificación compatibles con el grafo de cómputo de PyTorch, lo que permite calcularlas de forma diferenciable y acumulada a lo largo de los lotes sin gestión manual. Se emplea `BinaryAUROC`, `BinaryRecall` y `BinaryF1Score`.

Scikit-learn se emplea en la fase de evaluación final sobre el conjunto de prueba para generar el informe de clasificación por clase (precisión, *recall* y F1), la matriz de confusión y la tabla de anticipación segmentada por `cross_horizon`.

4.6 Contenerización: Docker

El entorno completo del *pipeline* se distribuye como imagen Docker basada en `ultralytics/ultralytics:latest`, que incluye PyTorch con soporte CUDA, YOLO, ByteTrack y OpenCV. Las dependencias de los pasos 2 y 3 (PyG, Lightning, TorchMetrics, scikit-learn, Streamlink, FFmpeg) se instalan en capas adicionales del Dockerfile. Esto garantiza que el *pipeline* es completamente reproducible en cualquier máquina con Docker y acceso a GPU NVIDIA, sin dependencias del sistema operativo anfitrión.

La Figura 4.3 ilustra la arquitectura general de Docker: el cliente envía órdenes al *daemon*, que gestiona imágenes, contenedores y volúmenes, y se comunica con registros remotos como Docker Hub para obtener imágenes base.

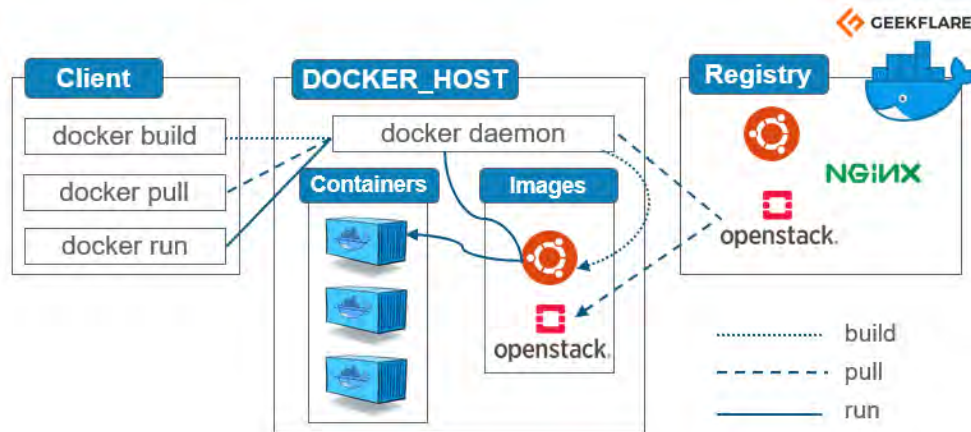


Figura 4.3: Arquitectura de Docker: cliente, *daemon* y registro
(fuente: <https://geekflare.com/es/docker-architecture/>).

4.7 Control de versiones: Git y GitHub

El desarrollo del proyecto se ha gestionado mediante **Git** como sistema de control de versiones distribuido y **GitHub** como plataforma de alojamiento remoto del repositorio. Git permite registrar el historial completo de cambios del código, revertir a versiones anteriores y trabajar con ramas independientes para experimentar sin afectar al código estable. GitHub añade una capa de respaldo remoto y facilita la trazabilidad del trabajo mediante el historial de *commits*.

Durante el proyecto se han empleado ramas separadas para el desarrollo de nuevas funcionalidades del *pipeline* (captura, etiquetado, criba) y para los experimentos de entrenamiento del modelo GNN, fusionándolas en la rama principal una vez validadas. Esta práctica ha permitido mantener en todo momento una versión operativa del sistema mientras se introducían mejoras de forma incremental.

La Figura 4.4 muestra la página principal del repositorio remoto. En ella se aprecia la estructura de directorios del proyecto: el código del *pipeline* (captura, etiquetado, criba y entrenamiento) se organiza bajo `gitTFG/codigofinal/`, mientras que la memoria LaTeX reside en `memoria/`. El archivo `README.md` describe los requisitos y los pasos necesarios para reproducir los experimentos.

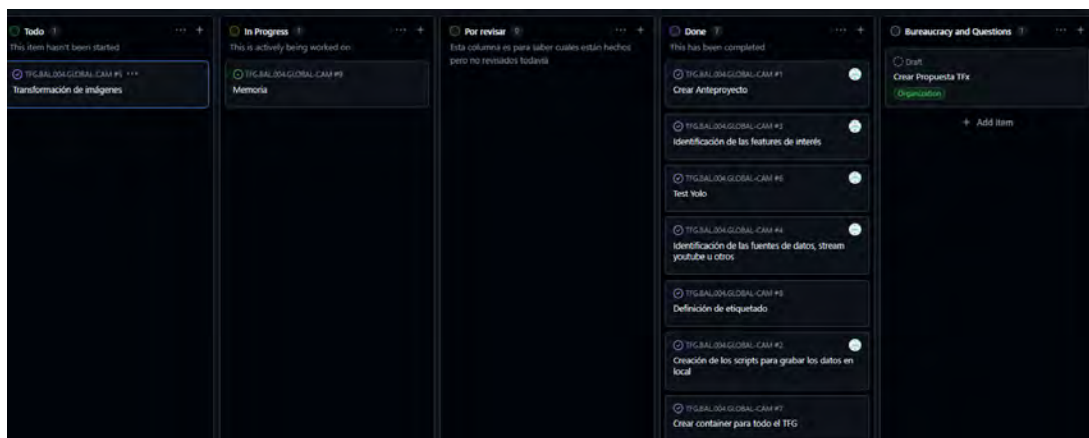


Figura 4.4: Página principal del repositorio en GitHub
(<https://github.com/invett/TFG.BAL.004.GLOBAL-CAM>), con la estructura de directorios del proyecto.

La Figura 4.5 recoge el historial de *commits* del repositorio. Cada entrada refleja un hito concreto del desarrollo: incorporación de nuevas cámaras al dataset, cambios en la arquitectura del modelo, ajuste de hiperparámetros o correcciones en el *pipeline*. El historial permite reconstruir el estado exacto del código en cualquier momento del proyecto y verificar qué modificaciones se introdujeron entre experimentos consecutivos.

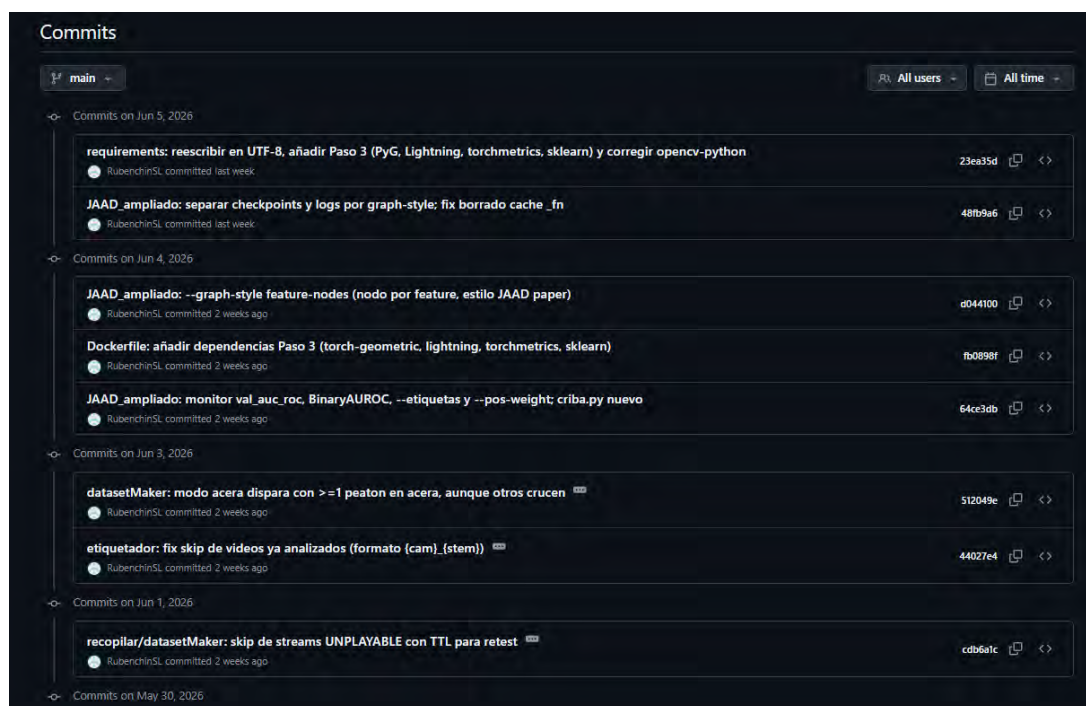


Figura 4.5: Historial de *commits* del proyecto en GitHub, donde cada entrada corresponde a un cambio significativo en el código o la configuración del sistema.

La Figura 4.6 muestra la wiki del repositorio, empleada como cuaderno de bitácora interno durante el desarrollo. En ella se han documentado las decisiones de diseño más relevantes, los resultados intermedios de los experimentos y los pasos de instalación del entorno. Al estar integrada en el propio repositorio, la wiki evoluciona junto al código y queda vinculada al historial de cambios del proyecto.

Home
Ruben edited this page 2 weeks ago · 2 revisions

Sistema de Estimación de Intenciones - Fase Técnica

Bienvenido a la documentación técnica del repositorio. Este proyecto implementa herramientas para la obtención, procesamiento y etiquetado automático de secuencias de vídeo destinadas al análisis de la intención de cruce de peatones.

El flujo de trabajo se divide en tres pasos, más un orquestador multicámara:

Paso 1	Captura	→ datasetMaker.py	grabación automática de cruces en streams
Paso 1b	Orquestador	→ recopilar.py	lanza datasetMaker en cadena para varias cámaras
Paso 2	Etiquetado	→ etiquetador.py	CSV de features por peatón y frame
Paso 3	Entrenamiento	→ JAAD_ampliado.py	GNN espacio-temporal (PyTorch Geometric + Lightning)

Módulos Operativos

Módulo	Función Principal	Tecnología
VideoDownloader	Descarga de clips desde YouTube / EarthCam.	Python + yt-dlp
GeneradorDataset	Captura de streams en vivo y auto-grabación de cruces o acera.	YOLO + ByteTrack + OpenCV
Orquestador	Lanza el GeneradorDataset en varias cámaras con timeout, skip y reintentos.	subprocess + threading
Etiquetador	Generación automática del CSV de features (formato JAAD).	YOLO + ByteTrack
JAAD_ampliado	Entrenamiento de la GNN sobre el CSV de features.	PyTorch Geometric + Lightning
TransformadorVideos	Corrección de perspectiva y vista cenital interactiva.	OpenCV + NumPy

Pages 14

Navegación Técnica

- Inicio

Entorno y Reproducibilidad

- Entorno de Ejecución
 - Docker (recomendado) y Python local
 - Versiones exactas de PyTorch, PyG y Lightning
- Guía de Reproducibilidad
 - Pipeline completo paso a paso
 - Árbol de artefactos esperado

Código Final — Pipeline TFG

Paso 1 — Captura

- Generador de Dataset
 - Captura de streams en vivo
 - Máscaras, tracking y auto-grabación
- Orquestador Multicámara
 - Automatización de múltiples cámaras
 - Timeout, reintentos y segunda vuelta

Figura 4.6: Wiki del repositorio en GitHub con la documentación interna del proyecto: decisiones de diseño, resultados de experimentos y guía de instalación.

Capítulo 5

Diseño del sistema: captura, etiquetado y dataset

Este capítulo describe el diseño e implementación del *pipeline* de obtención de datos, que constituye la contribución de ingeniería central de este trabajo. El *pipeline* se organiza en tres pasos secuenciales: captura de secuencias de vídeo desde *streams* de cámaras públicas (Paso 1), extracción automática de *features* y etiquetado por trayectoria (Paso 2), y preparación del dataset para el entrenamiento de la GNN (Paso 3). Los Pasos 1 y 2 se describen en detalle en este capítulo; el Paso 3 se aborda en el capítulo 6.

5.1 Visión general del *pipeline*

Paso	Componente y salida
1	<code>datasetMaker.py</code> / <code>recopilar.py</code> → clips .mp4 en <code>datos/1_crudo/secuencias/</code>
2	<code>etiquetador.py</code> → CSV de <i>features</i> en <code>datos/2_etiquetado/etiquetas/</code>
2b	<code>criba.py</code> → CSV balanceado en <code>datos/2_etiquetado/etiquetas/</code>
3	<code>JAAD_ampliado.py</code> → grafos PyG y <i>checkpoint</i> del modelo

El código de cada paso reside en `gitTFG/codigofinal/`. Los datos se almacenan en `datos/`, con subdirectorios por etapa. Todas las rutas se resuelven a través de `rutas.py`, un módulo centralizado que ancla las rutas al directorio del propio archivo (`__file__`) en lugar del directorio de trabajo actual; esto garantiza que los *scripts* funcionan correctamente independientemente del directorio desde el que se invoquen.

La figura 5.1 ilustra el flujo completo del *pipeline* en sus tres etapas principales.

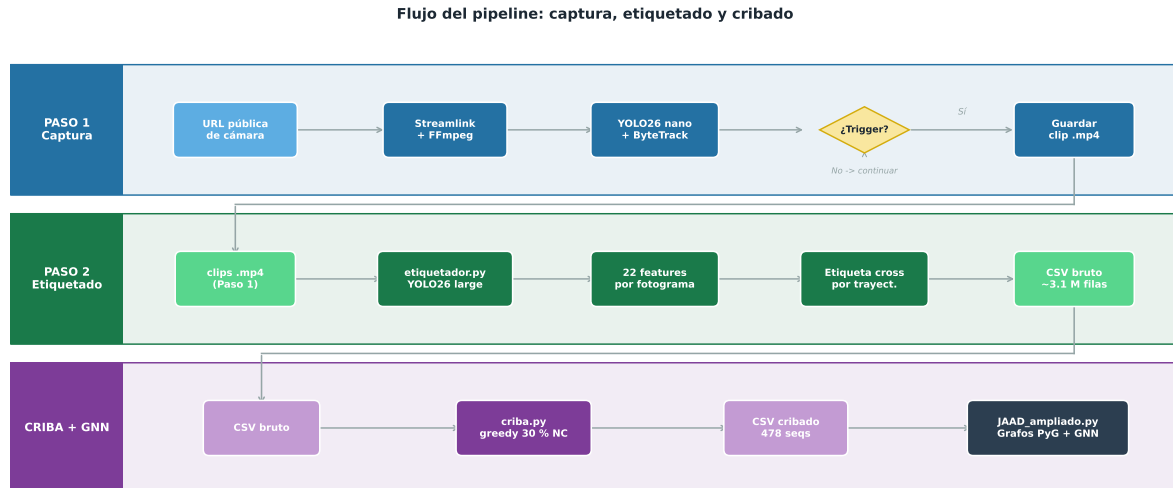


Figura 5.1: Flujo del *pipeline*: captura (Paso 1), etiquetado (Paso 2) y cribado + entrenamiento GNN (Paso 3).

5.2 Paso 1: Captura de secuencias (**datasetMaker.py**)

`datasetMaker.py` es el motor de captura. Se conecta a un *stream* de vídeo público, ejecuta detección y seguimiento de peatones en tiempo real, y graba automáticamente clips cortos cuando detecta un evento de interés.

5.2.1 Configuración de zonas de interés (ROI)

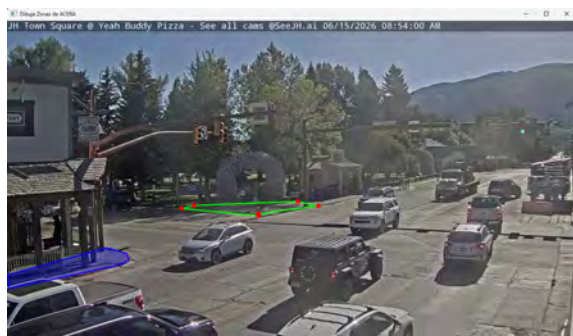
Antes de la primera captura de una cámara nueva, el usuario dibuja con el ratón cuatro tipos de zona sobre un fotograma congelado del *stream*, mediante una interfaz OpenCV interactiva:

Tabla 5.1: Tipos de zona ROI configurables por cámara.

Fase	Zona	Archivo	Obligatoria
1	Acera (zona segura)	<code>acera.json</code>	Sí
2	Calzada (zona cruce)	<code>carretera.json</code>	Sí
3	Semáforos	<code>semaforos.json</code>	No
4	Paso de cebra	<code>cebra.json</code>	No

Las máscaras se guardan en formato JSON en `datos/1_crudo/mascaras/{NN_camara}/` y se reutilizan en todas las sesiones posteriores de la misma cámara. Las fases 3 y 4 son opcionales: si la cámara no dispone de semáforo visible o de paso de cebra, se puede omitir el dibujo y las *features* correspondientes tomarán valores neutros.

La figura 5.2 muestra las cuatro etapas de configuración de máscaras ROI sobre la cámara de Jackson Town Square (Wyoming), y la vista global con todas las zonas superpuestas.



Fase 1 · Acera



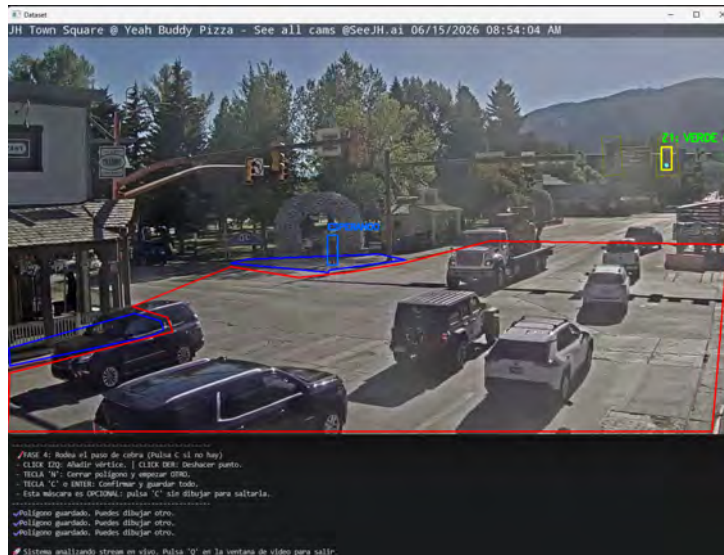
Fase 2 · Calzada



Fase 3 · Semáforos



Fase 4 · Paso de cebra



Vista global · Todas las zonas ROI superpuestas

Figura 5.2: Etapas de configuración de máscaras ROI (OpenCV) sobre la cámara 02 (Jackson Town Square, Wyoming): acera, calzada, semáforos, paso de cebra y vista global con todas las zonas activas.

5.2.2 Modos de grabación

El sistema ofrece dos modos de grabación complementarios, seleccionables mediante el argumento `--modo`:

Modo cruce (predeterminado). El disparador se activa cuando un peatón con origen en la zona de acera permanece en la zona de calzada durante al menos 1 segundo ininterrumpido (filtro anti-falsos-positivos). Se guarda un clip que incluye los `SEGUNDOS_PRE_TRIGGER` anteriores al cruce (tomados de un *buffer* circular en RAM) más los `SEGUNDOS_POST_TRIGGER` posteriores. El *label* resultante es `crossing`.

Modo acera. El disparador se activa cuando al menos un peatón permanece en la zona de acera durante 5 segundos o más sin cruzar, seguido de un período de *cooldown* de 30 segundos para evitar grabar repetidamente la misma escena. El *label* resultante es `not-crossing`. Este modo se diseñó específicamente para recopilar ejemplos de la clase minoritaria y equilibrar el dataset sin depender de eventos de cruce.

5.2.3 Filtro de personas en vehículos

Para evitar que ciclistas, motoristas o personas transportadas en vehículos sean clasificados como peatones, el sistema comprueba el solapamiento geométrico entre la caja del peatón detectado y las cajas de los vehículos presentes en el mismo fotograma. Si más del 50 % del área del peatón se solapa con el área expandida de un vehículo, la detección se descarta.

5.2.4 Grabación sin *heads-up display*

Los clips se graban sin el Heads-Up Display (HUD) (cajas de detección, etiquetas, zonas coloreadas) que se muestra en la ventana de visualización en tiempo real. El *overlay* se dibuja únicamente sobre la imagen de pantalla, no sobre los fotogramas escritos en disco. Esto garantiza que el etiquetador (Paso 2) recibe fotogramas limpios, no contaminados por artefactos del propio sistema.

5.3 Orquestador multicámara (`recopilar.py`)

`recopilar.py` automatiza la captura en múltiples cámaras lanzando instancias de `datasetMaker.py --headless` en secuencia, con gestión automática de *timeouts*, reintentos y sesiones múltiples. Está diseñado para ejecutarse de forma desatendida durante horas o días, completando progresivamente el objetivo de secuencias por cámara.

Las decisiones de diseño más relevantes del orquestador son:

Timeout por inactividad. Si transcurren 2 minutos sin que se guarde ninguna nueva secuencia en la carpeta de la cámara activa, el proceso se termina y se pasa a la siguiente cámara. Esto evita que el orquestador quede bloqueado en una cámara con poco tráfico peatonal.

Skip de cámaras ya completadas. Al inicio de cada cámara se comprueba si la carpeta ya contiene el número objetivo de secuencias; si es así, se salta sin lanzar ningún proceso.

Gestión de *streams* no disponibles con TTL. Si `datasetMaker.py` detecta que el *stream* está caído, el orquestador marca esa cámara como no disponible con una marca de tiempo y no vuelve a intentarla hasta que transcurra el TTL configurado (30 minutos por defecto). Este mecanismo evita que cámaras temporalmente *offline* bloqueen el avance de las demás.

Vueltas de reintento configurables. Con `--vueltas 0` el orquestador ejecuta un bucle infinito hasta que todas las cámaras completen su objetivo, lo que lo hace adecuado para sesiones de captura no supervisadas de larga duración.

La figura 5.3 resume la lógica del orquestador: el camino principal de ejecución y las condiciones de salto o fin de sesión.

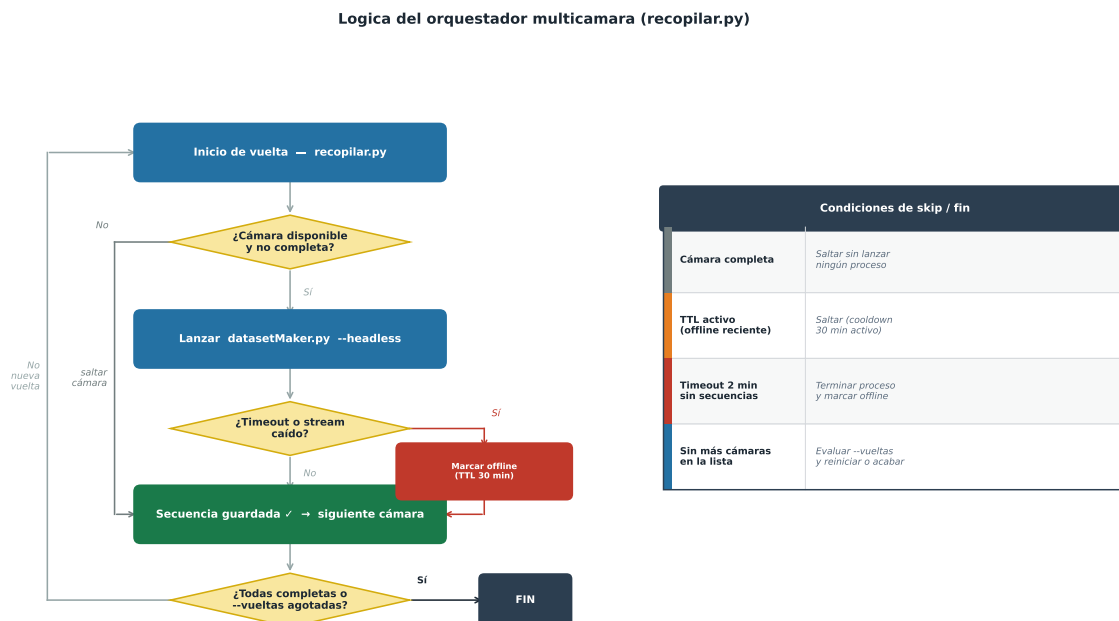


Figura 5.3: Lógica del orquestador multicámara (`recopilar.py`): flujo principal y condiciones de *skip* / fin de sesión.

5.4 Paso 2: Etiquetado automático (`etiquetador.py`)

`etiquetador.py` re-procesa los clips `.mp4` del Paso 1 y produce el Comma-Separated Values (CSV) de *features* que alimenta la GNN del Paso 3. Su *pipeline* interno consta de tres etapas:

1. **Detección y seguimiento:** `model.track()` aplica YOLO26 + ByteTrack sobre cada fotograma para detectar peatones, vehículos, semáforos y mascotas con identificadores estables.
2. **Cálculo de *features*:** para cada peatón detectado en cada fotograma se computan las 20 columnas del CSV.
3. **Etiquetado por trayectoria:** en una segunda pasada sobre la trayectoria completa de cada peatón se asigna la etiqueta `cross`.

Una fila del CSV equivale a un peatón en un fotograma.

5.4.1 Esquema del CSV (22 columnas)

El CSV adopta el mismo nombre y valores de columna que el dataset JAAD [2], de modo que el código de entrenamiento puede consumirlo sin modificaciones. Las 22 columnas se dividen en tres grupos:

Identificadores y metadatos (8 columnas): `video`, `frame`, `person` (ID de ByteTrack), `camera`, `fps`, `cx`, `cy` (centroide en píxeles), `hud_dirty` (indica si el clip tiene *overlay* superpuesto).

***Features* compatibles con JAAD (5 + etiqueta):**

***Features* propias (8 columnas):**

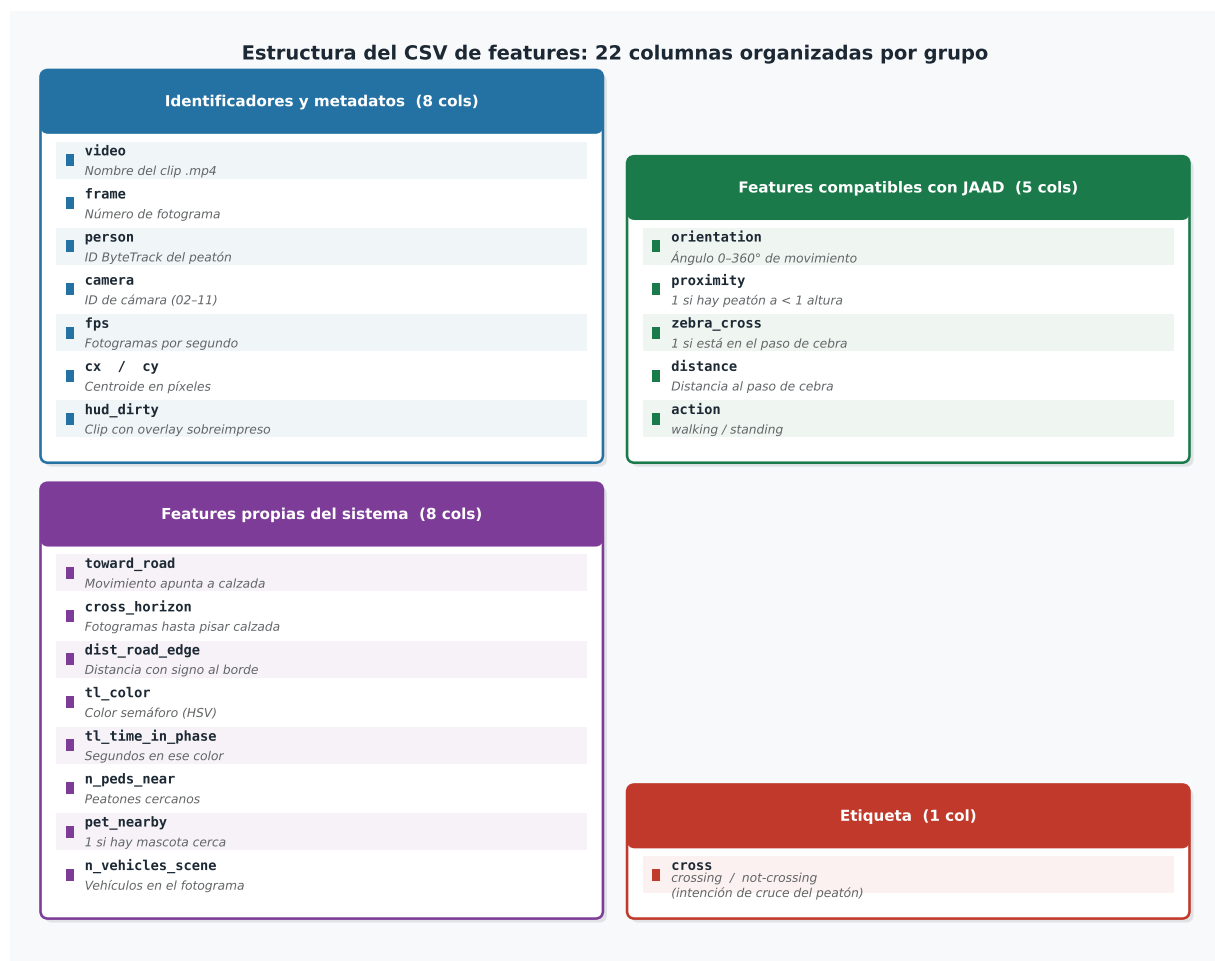
La figura 5.4 muestra las 22 columnas del CSV organizadas por grupo.

Tabla 5.2: *Features* compatibles con JAAD.

<i>Feature</i>	Tipo	Descripción
orientation	numérica	Ángulo 0–360° de la dirección de movimiento
proximity	binaria	1 si hay otro peatón a < 1 altura de caja
zebra_cross	binaria	1 si el peatón está dentro del paso de cebra
distance	numérica	Distancia al paso de cebra
action	categorica	walking / standing
cross	etiqueta	crossing / not-crossing

Tabla 5.3: *Features* propias del sistema.

<i>Feature</i>	Tipo	Descripción
toward_road	binaria	1 si el vector de movimiento apunta a la calzada
cross_horizon	numérica	Fotogramas hasta que el peatón pisa la calzada
dist_road_edge	numérica	Distancia con signo al borde de la calzada
tl_color	categorica	Color del semáforo más cercano
tl_time_in_phase	numérica	Segundos que el semáforo lleva en ese color
n_peds_near	numérica	Número de peatones cercanos
pet_nearby	binaria	1 si hay un perro o gato cerca del peatón
n_vehicles_scene	numérica	Número de vehículos en el fotograma

Figura 5.4: Estructura del CSV de *features*: 22 columnas organizadas en cuatro grupos (metadatos, compatibles JAAD, propias y etiqueta).

5.4.2 Decisiones de diseño relevantes

Sustitución de `attention` por `toward_road`. JAAD incluye una *feature attention* basada en la orientación de la cabeza del peatón. En cámaras públicas con peatones distantes, la estimación de pose facial produce un 98 % de valores iguales (la pose no es fiable a baja resolución), convirtiendo la *feature* en constante. Se sustituye por `toward_road`, que proyecta el vector de movimiento sobre la máscara de calzada: da información equivalente de manera más robusta. `JAAD_ampliado.py` tolera la ausencia de *attention* mediante `df.get()`.

Etiqueta por trayectoria, no por fotograma. La etiqueta `cross` refleja la *intención* del peatón, no su estado instantáneo. Vale `crossing` si el peatón, habiendo partido de la zona de acera, llega a entrar en la calzada en algún momento de la secuencia. Esta asignación se realiza en una segunda pasada sobre la trayectoria completa.

***Skip* de vídeos ya analizados.** Al arrancar, el etiquetador detecta automáticamente qué clips ya aparecen en los CSV existentes y pregunta si desea saltarlos, permitiendo ejecutar ciclos de captura y etiquetado de forma incremental.

5.5 Dataset resultante

5.5.1 Cámaras y cobertura geográfica

El dataset se construyó a partir de diez cámaras de tráfico públicas distribuidas en cinco continentes:

Tabla 5.4: Cámaras del dataset.

ID	Nombre / Localización	Continente
02	Jackson Town Square, Wyoming (EEUU)	América del Norte
03	Main Street Livecam, Canmore (Canadá)	América del Norte
04	Nevskiy Avenue, San Petersburgo (Rusia)	Europa
05	North Conway, New Hampshire (EEUU)	América del Norte
06	Tanukikoji 8-chome, Sapporo (Japón)	Asia
07	Walworth Road, Londres (Reino Unido)	Europa
08	Bourbon Street, Nueva Orleans (EEUU)	América del Norte
09	Abbey Road, Londres (Reino Unido)	Europa
10	Shinjuku – , Tokio (Japón)	Asia
11	Bondi Aussie Bar & Grill, Sídney (Australia)	Oceanía

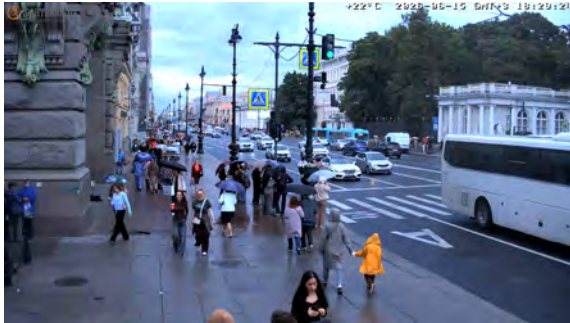
La figura 5.5 muestra un fotograma representativo de cada una de las diez localizaciones, ilustrando la diversidad geográfica y visual del dataset.



02 · Jackson Town Square, Wyoming



03 · Main Street, Canmore



04 · Nevskiy Avenue, San Petersburgo



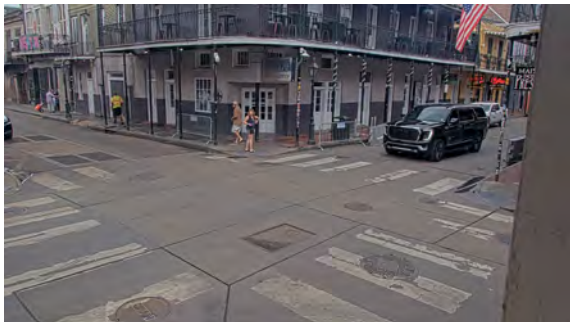
05 · North Conway, New Hampshire



06 · Tanukikoji, Sapporo



07 · Walworth Road, Londres (test)



08 · Bourbon Street, Nueva Orleans (test)



09 · Abbey Road, Londres



10 · Shinjuku, Tokio



11 · Bondi, Sídney

Figura 5.5: Fotogramas representativos de las diez cámaras del dataset, ordenados por ID. Las cámaras 07 y 11 (fondo naranja) corresponden al conjunto de prueba.

5.5.2 Estadísticas del CSV bruto y cribado

El CSV bruto `etiquetas_ALL.csv`, generado directamente por el etiquetador, contiene **3,10 millones de filas** sobre 10 cámaras y aproximadamente 9.800 pistas únicas. El dataset bruto presenta un fuerte desbalance: aproximadamente el 78 % de las pistas son `crossing` y el 22 % son `not-crossing`.

Para reducir este desbalance, el *script* `criba.py` aplica un algoritmo *greedy* que descarta secuencias completas por orden de menor porcentaje de `not-crossing`, hasta que la proporción global de pistas `not-crossing` alcanza el objetivo del 30 %. El CSV cribado `etiquetas_ALL_criba.csv` contiene:

Tabla 5.5: Estadísticas del CSV cribado.

Estadística	Valor
Filas totales	2.350.996
Secuencias únicas	478
Cámaras	10
Proporción <code>not-crossing</code> (a nivel pista)	~30 %

La figura 5.6 ilustra el efecto del cribado sobre la distribución de clases.

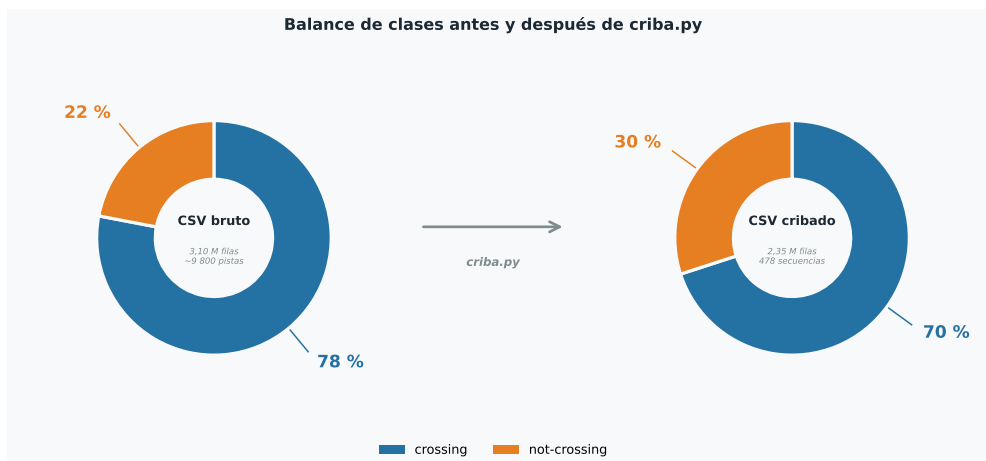


Figura 5.6: Balance de clases antes y después de `criba.py`: el porcentaje de pistas `not-crossing` sube del 22 % al 30 %.

5.5.3 Split de entrenamiento y prueba

El dataset se divide en entrenamiento y prueba a nivel de cámara completa, no de fila individual ni de secuencia. Esta decisión evita el *data leakage* geográfico: si el mismo fondo de imagen (la misma cámara) aparece en ambos subconjuntos, la red puede aprender a reconocer la escena en lugar de la intención del peatón. Separar cámaras completas garantiza que el test evalúa genuina generalización a escenas no vistas.

El 20 % de las cámaras se asignan al test (2 de 10) y el 80 % al entrenamiento (8 de 10):

Entrenamiento — 8 cámaras, 439 secuencias, 2.277.041 filas: cámaras 02 (Wyoming), 03 (Canmore), 04 (San Petersburgo), 05 (New Hampshire), 06 (Sapporo), 09 (Abbey Road), 10 (Shinjuku) y 11 (Sídney).

Prueba — 2 cámaras, 39 secuencias, 73.955 filas: cámara 07 (Walworth Road, Londres) y cámara 08 (Bourbon Street, Nueva Orleans). El modelo no vio ningún fotograma de estas dos cámaras durante el entrenamiento.

Capítulo 6

Modelo GNN y metodología de entrenamiento

Este capítulo describe el modelo de red neuronal de grafos espacio-temporal implementado en `JAAD_ampliado.py`, la construcción del dataset de grafos a partir del CSV de *features*, la arquitectura de la red y las decisiones metodológicas tomadas en cuanto a función de pérdida, métricas de monitorización y configuración del entrenamiento.

6.1 Construcción de grafos (**DatasetAmpliado**)

El CSV de *features* producido por el etiquetador es una tabla plana de filas (peatón $\times$ fotograma). Para alimentar una GNN, esta tabla se convierte en un dataset de objetos `Data` de PyG, donde cada objeto representa un grafo espacio-temporal.

Cada grafo corresponde a un **tripleto de fotogramas consecutivos** ($t - 1, t, t + 1$) de un mismo vídeo. La GNN opera sobre este triplete y emite una predicción de intención de cruce para cada peatón del fotograma central t . Los fotogramas $t - 1$ y $t + 1$ proporcionan contexto temporal sin que sea necesario mantener estado recurrente entre grafos.

El sistema implementa dos estilos de construcción de grafos seleccionables mediante `--graph-style`.

6.1.1 Estilo **interaction**

En el estilo *interaction*, cada peatón presente en cada fotograma del triplete es un nodo. El vector de atributos de cada nodo es el vector de *features* codificado de dimensión $C_{IN} = 20$.

Las aristas son de dos tipos:

Tabla 6.1: Tipos de aristas en el grafo de estilo *interaction*.

Tipo	Conecta	Qué propaga
Espacial (intra-fotograma)	Todos los peatones del mismo fotograma	Contexto social
Temporal (inter-fotograma)	El mismo peatón entre fotogramas consecutivos	Dinámica individual

Las aristas son bidireccionales. La etiqueta y el `cross_horizon` se asignan exclusivamente a los nodos del fotograma central (`central_index`).

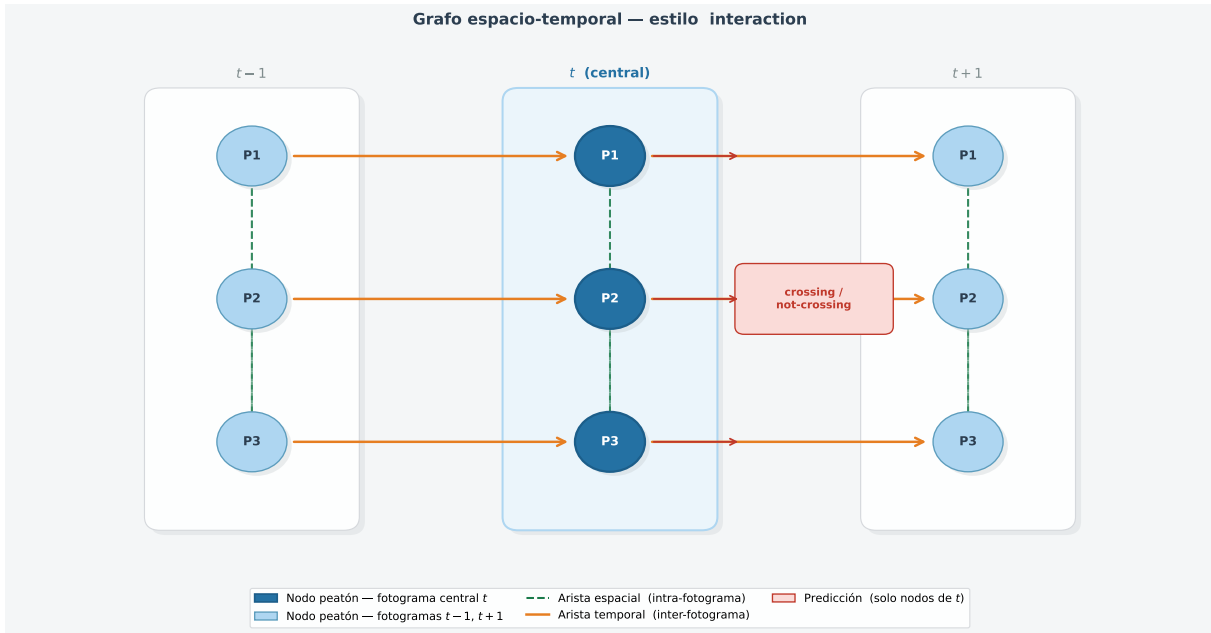


Figura 6.1: Grafo espacio-temporal estilo *interaction*. Tres fotogramas consecutivos ($t-1, t, t+1$) con tres peatones por fotograma. Las aristas espaciales (verde discontinuo) conectan peatones del mismo fotograma; las aristas temporales (naranja) conectan el mismo peatón entre fotogramas consecutivos. Solo los nodos del fotograma central t emiten predicción.

6.1.2 Estilo *feature-nodes*

En el estilo *feature-nodes*, cada peatón se descompone en **13 nodos**: un nodo peatón y 12 nodos *feature* (uno por grupo de *features*). Los nodos *feature* codifican el valor de la *feature* correspondiente junto con un identificador *one-hot* del tipo de *feature*. La dimensión del vector de nodo es $C_{IN} = 21$.

Las aristas intra-fotograma conectan el nodo peatón con sus 12 nodos *feature* (estructura en estrella), más las conexiones sociales entre nodos peatón de distintos peatones. Las aristas temporales conectan el nodo peatón del fotograma t con el del mismo peatón en $t+1$, y los nodos *feature* del mismo tipo entre fotogramas consecutivos.

La motivación de este estilo es que expone a la GNN cada *feature* individualmente, permitiéndole aprender qué señales son discriminativas y propagar esa información de forma selectiva, en lugar de tratar el vector de 20 *features* como un bloque opaco.

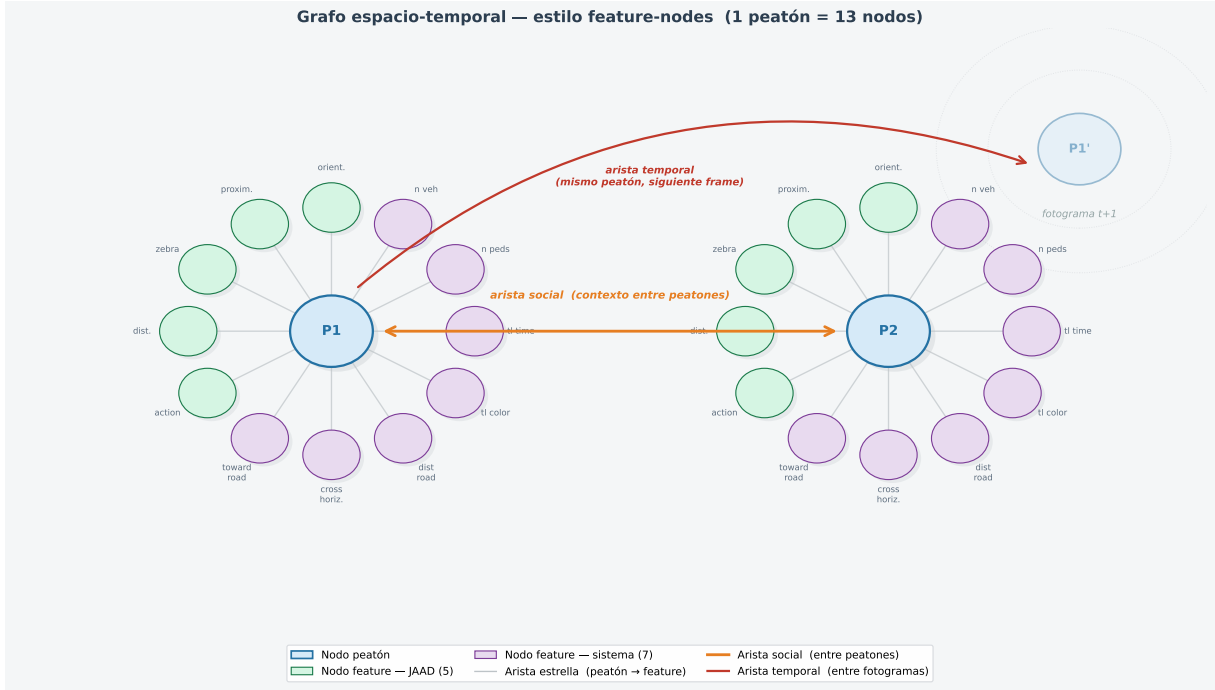


Figura 6.2: Grafo espacio-temporal estilo feature-nodes. Cada peatón se descompone en 13 nodos: un nodo peatón central y 12 nodos *feature* (5 compatibles con JAAD en verde, 7 propios del sistema en morado) organizados en estructura de estrella. La arista social (naranja) conecta los nodos peatón de distintos peatones; la arista temporal (rojo) conecta el nodo peatón con su instancia en el fotograma siguiente.

6.2 Arquitectura del modelo

El modelo se implementa en tres módulos apilados, cuya composición se ilustra en la figura 6.3.

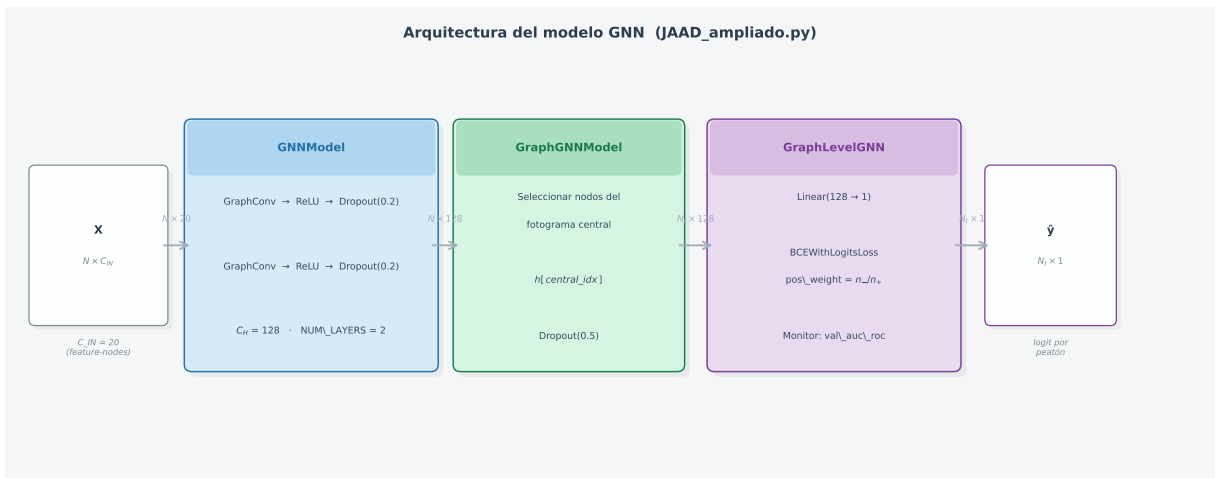


Figura 6.3: Arquitectura del modelo GNN. El tensor de entrada ($N \times C_{IN}$) atraviesa dos capas GraphConv con $C_{HIDDEN} = 128$ unidades (GNNModel), se filtran los nodos del fotograma central t (GraphGNNModel) y una capa lineal con BCEWithLogitsLoss produce un logit por peatón (GraphLevelGNN).

6.2.1 GNNModel — núcleo convolucional

Dos capas GraphConv con $C_{HIDDEN} = 128$ unidades, separadas por ReLU y *dropout* (0,2). Dos capas permiten que cada nodo integre información de vecinos a distancia 2, lo que en el grafo espacio-temporal equivale a nodos dos saltos temporales o sociales alejados.

6.2.2 GraphGNNModel — cabeza de clasificación

Extrae los *embeddings* de los nodos del fotograma central (*central_index*). Los nodos de $t-1$ y $t+1$ contribuyen a través de las aristas temporales pero no generan predicciones propias. Un *dropout* (0,5) previo a la capa lineal actúa como regularizador.

6.2.3 GraphLevelGNN — módulo Lightning

Tabla 6.2: Configuración del módulo Lightning.

Componente	Configuración
Pérdida	BCEWithLogitsLoss ($\text{pos_weight}=n_{neg}/n_{pos}$)
Optimizador	AdamW ($\text{lr}=1\text{e-}3$)
<i>Scheduler</i>	ReduceLROnPlateau ($\text{patience}=1$) sobre <i>val_loss</i>
Métricas validación	BinaryRecall, BinaryF1Score, BinaryAUROC

Los hiperparámetros globales fijados son: $C_{IN} = 20$ (21 en feature-nodes), $C_{HIDDEN} = 128$, $\text{NUM_LAYERS} = 2$, *dropout* convolucional 0,2, *dropout* lineal 0,5, tamaño de lote 100, y un máximo de 50 épocas.

6.3 Tratamiento del desbalance de clases (*pos_weight*)

El dataset cribado presenta aproximadamente un 70% de ejemplos *crossing* y un 30% de *not-crossing*. Con este desbalance, la función de pérdida estándar lleva al modelo hacia la predicción constante de *crossing* (modo colapso). Para corregirlo, BCEWithLogitsLoss acepta un escalar *pos_weight* calculado automáticamente sobre el split de entrenamiento como:

$$\text{pos_weight} = \frac{n_{neg}}{n_{pos}} \quad (6.1)$$

Con el dataset cribado al 30% de *not-crossing*, el valor automático es aproximadamente 0,43. Este valor puede sobrescribirse por CLI para experimentar con distintos *trade-offs*, como se analiza en el capítulo 7.

6.4 Métricas de evaluación y monitorización

6.4.1 AUC-ROC (métrica principal)

El AUC-ROC evalúa la capacidad del modelo para ordenar correctamente los ejemplos positivos por encima de los negativos a lo largo de todos los posibles umbrales de decisión. Un modelo que predice siempre la clase mayoritaria obtiene AUC-ROC= 0,5; un modelo perfecto obtiene 1,0. Esta propiedad lo hace resistente al desbalance de clases, a diferencia de la *accuracy* o del *F1-crossing*, que se maximizan trivialmente prediciendo siempre *crossing*.

Por este motivo, *EarlyStopping* y *ModelCheckpoint* se configuran para monitorizar *val_auc_roc*.

La figura 6.4 muestra las curvas de convergencia reales obtenidas durante el entrenamiento con ambos estilos de grafo.

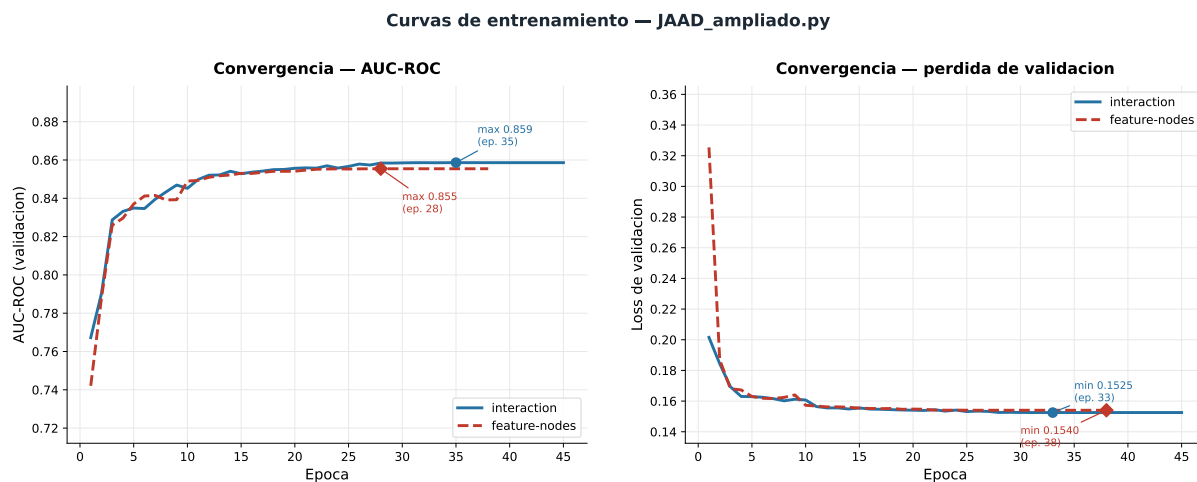


Figura 6.4: Curvas de entrenamiento reales de JAAD_ampliado.py. Panel izquierdo: AUC-ROC de validación por época para el estilo *interaction* (azul, línea continua) y *feature-nodes* (rojo, línea discontinua). Panel derecho: pérdida de validación correspondiente. Los marcadores indican la época óptima seleccionada por ModelCheckpoint.

6.4.2 Recall por clase

Se reportan los *recalls* de *crossing* y *not-crossing* por separado. Para el sistema de seguridad vial, los dos tipos de error tienen consecuencias distintas: un falso negativo en *crossing* (el sistema no alerta cuando debería) es el error más peligroso; un falso positivo en *crossing* (falsa alarma) puede inutilizar el sistema acumulado.

6.4.3 Anticipación por horizonte (*cross_horizon*)

El *recall* segmentado por *buckets* de *cross_horizon* mide con cuánta antelación el modelo predice correctamente la intención:

Tabla 6.3: *Buckets* de horizonte de anticipación (a 30 fps).

<i>Bucket</i>	Interpretación
En cruce (horizonte ≈ 0)	El peatón ya está en la calzada
1–5 fotogramas ($<0,2$ s)	Inminente
6–15 fotogramas ($0,2$ – $0,5$ s)	Muy próximo
16–30 fotogramas ($0,5$ – $1,0$ s)	Próximo
>30 fotogramas ($>1,0$ s)	Margen de reacción real

6.5 Configuración del entrenamiento

```
ModelCheckpoint(monitor="val_auc_roc", mode="max")
```

```
EarlyStopping(monitor="val_auc_roc", patience=10, mode="max")
```

ModelCheckpoint guarda el *checkpoint* de la época con mejor AUC-ROC en validación. EarlyStopping detiene el entrenamiento si el AUC-ROC en validación no mejora durante 10 épocas consecutivas, evitando el sobreajuste sin necesidad de fijar manualmente el número de épocas. El

checkpoint guardado se recarga automáticamente al final del entrenamiento para la evaluación en test, garantizando que los resultados reportados corresponden siempre al mejor estado del modelo.

Capítulo 7

Pruebas y resultados

Este capítulo presenta los resultados obtenidos a lo largo del proceso de desarrollo. El entrenamiento se ejecutó en cinco iteraciones sucesivas, cada una motivada por un diagnóstico de la anterior. Las métricas reportadas en todas las ejecuciones corresponden al conjunto de prueba (cámaras 07 y 08), evaluado con el *checkpoint* de mejor AUC-ROC en validación.

7.1 Ejecución 1: modo colapso sin `pos_weight`

La primera ejecución utilizó el estilo `interaction` sin ningún mecanismo de corrección del desbalance de clases. El resultado fue un **modo colapso**: el modelo aprendió a predecir siempre `crossing=1`, la clase mayoritaria.

Con el $\sim 70\%$ de ejemplos `crossing` en el dataset, esta estrategia trivial produce una *accuracy* aparente de 0,70, un *recall* de `crossing` de 1,0 y un *recall* de `not-crossing` de 0,0. El modelo no aprendió ninguna representación útil.

Causa raíz. Sin penalización adicional para la clase minoritaria, el gradiente de la función de pérdida empuja al modelo hacia la predicción constante de la clase mayoritaria, que minimiza la pérdida sin necesidad de aprender características discriminativas.

Corrección aplicada. Se introdujo `pos_weight = nneg/npos` en `BCEWithLogitsLoss`, calculado automáticamente sobre el split de entrenamiento.

7.2 Ejecución 2: **EarlyStopping** sobre `val_f1`

Con `pos_weight` el modelo comenzó a aprender. Sin embargo, `EarlyStopping` y `ModelCheckpoint` monitorizaban `val_f1` (F1 de la clase `crossing`), que es máximo en la época 0 (predicción constante = F1-*crossing* máximo). El *callback* guardó el *checkpoint* de la época 0 y detuvo el entrenamiento prematuramente.

Tabla 7.1: Resultados de la ejecución 2.

Métrica	Valor
AUC-ROC	0,715
Recall <code>not-crossing</code>	0,292
Recall <code>crossing</code>	0,958
Macro F1	0,650
Anticipación >30f	92,2 %

Pese al error en la monitorización, el AUC-ROC de 0,715 confirmó que el `pos_weight` estaba funcionando. La corrección aplicada fue cambiar el monitor de ambos *callbacks* a `val_auc_roc`.

7.3 Ejecución 3: configuración de referencia (**interaction**)

Con monitor `val_auc_roc` y `pos_weight` automático ($\approx 0,226$ con el dataset cribado al 30%), el modelo convergió en 50 épocas sobre el estilo *interaction*.

Tabla 7.2: Resultados de la ejecución 3 (referencia).

Métrica	Valor
AUC-ROC	0,737
Recall not-crossing	0,335
Recall crossing	0,923
Macro F1	0,637
Verdaderos negativos (TN)	3.130
Anticipación >30 f	89,6 %

Esta ejecución establece la **línea base de referencia**. El modelo anticipa correctamente el cruce en casi 9 de cada 10 escenas con horizonte >1 segundo.

7.4 Ejecución 4: experimento con **pos_weight = 0,1**

Con el objetivo de incrementar el *recall* de *not-crossing*, se redujo manualmente `pos_weight` a 0,1.

Tabla 7.3: Resultados de la ejecución 4 frente a la ejecución 3.

Métrica	Valor	vs Ejec. 3
AUC-ROC	0,736	-0,001
Recall not-crossing	0,508	+0,173
Recall crossing	0,825	-0,098
Macro F1	0,622	-0,015
Anticipación >30 f	73,1 %	-16,5 pp

La hipótesis se confirmó parcialmente: el *recall* de *not-crossing* subió 17,3 pp. Sin embargo, la **anticipación temprana cayó 16,5 pp**, lo que significa que el modelo falla en advertir el cruce con más de 1 segundo de antelación en 1 de cada 4 escenas. Para un sistema de seguridad vial, ese coste es inaceptable. Esta ejecución documenta el *trade-off* fundamental de `pos_weight`: valores bajos favorecen la detección de *not-crossing* a expensas de la anticipación temprana. Esta ejecución se descartó.

7.5 Ejecución 5: estilo **feature-nodes** — mejor resultado

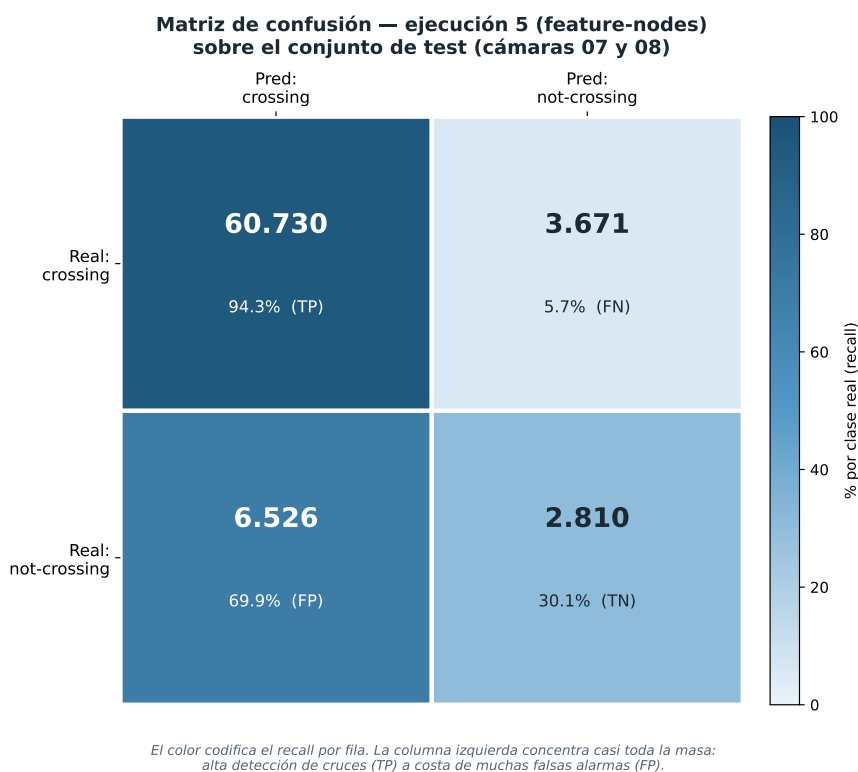
Se repitió la configuración de la ejecución 3 con el estilo *feature-nodes*: cada peatón se representa como 13 nodos en lugar de un único nodo con vector de 20 atributos. El entrenamiento convergió con parada temprana en la época 37.

El estilo *feature-nodes* mejora las dos métricas más importantes: el AUC-ROC sube 2,7 pp y la anticipación temprana sube 2,9 pp. La única contrapartida es una ligera bajada del *recall* de *not-crossing* (0,301 vs. 0,335), que sigue siendo aceptable dado que la prioridad es la detección anticipada de cruces.

La matriz de confusión de esta ejecución (Figura 7.1) ilustra el comportamiento del modelo sobre el conjunto de test: la masa de predicciones se concentra en la detección de cruces (TP), mientras que la alta proporción de falsos positivos (FP) refleja la tasa de falsas alarmas asociada al `pos_weight` automático.

Tabla 7.4: Resultados de la ejecución 5 (feature-nodes).

Métrica	Valor	vs Ejec. 3
AUC-ROC	0,764	+0,027
Recall not-crossing	0,301	−0,034
Recall crossing	0,943	+0,020
Macro F1	0,639	+0,002
Verdaderos negativos (TN)	2.810	—
Verdaderos positivos (TP)	60.730	—
Anticipación >30f	92,5 %	+2,9 pp
Épocas hasta convergencia	37	—
pos_weight (automático)	0,198	—

Figura 7.1: Matriz de confusión de la ejecución 5 sobre el conjunto de test. El color codifica el *recall* por clase real; los recuentos de FP y FN se derivan de los *recalls* reportados.

7.6 Tabla comparativa de todas las ejecuciones

Tabla 7.5: Comparativa de las cinco ejecuciones de entrenamiento.

E.	Estilo	Monitor	pw	AUC-ROC	Recall NC	Antic. >30f
1	interaction	val_f1	—	—	0,000	—
2	interaction	val_f1	auto	0,715	0,292	92,2 %
3	interaction	val_auc_roc	auto	0,737	0,335	89,6 %
4	interaction	val_auc_roc	0,1	0,736	0,508	73,1 %
5	feat.-nodes	val_auc_roc	auto	0,764	0,301	92,5 %

La Figura 7.2 representa gráficamente las dos métricas clave de la tabla anterior. Se observa cómo la ejecución 5 (feature-nodes) maximiza simultáneamente el AUC-ROC y la anticipación temprana, mientras que la ejecución 4 (con pos_weight reducido) sacrifica notablemente la anticipación.

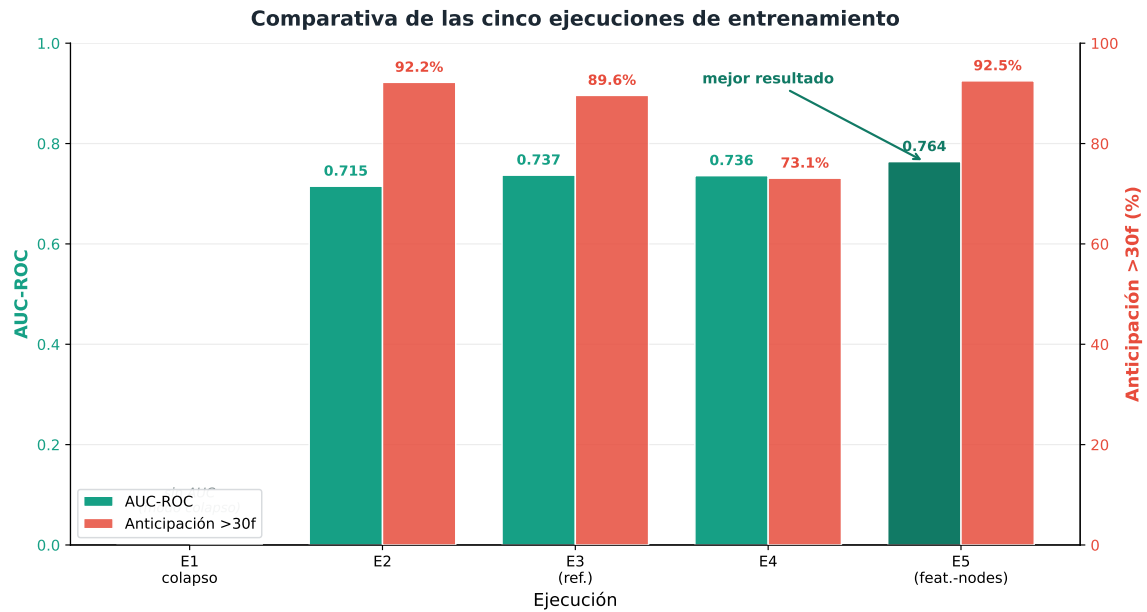


Figura 7.2: Comparativa de AUC-ROC (eje izquierdo) y anticipación >30 fotogramas (eje derecho) de las cinco ejecuciones. La ejecución 1 no produjo AUC-ROC válido por entrar en modo colapso.

7.7 Análisis de la anticipación por horizonte (ejecución 5)

Tabla 7.6: Recall de crossing por *bucket* de horizonte (ejecución 5).

<i>Bucket</i> de horizonte	Anticipación (<i>recall crossing</i>)
En cruce (horizonte ≈ 0)	$\sim 1,000$
1–5 fotogramas ($< 0,2$ s)	$\sim 0,980$
6–15 fotogramas (0,2–0,5 s)	$\sim 0,965$
16–30 fotogramas (0,5–1,0 s)	$\sim 0,950$
>30 fotogramas ($> 1,0$ s)	0,925

El modelo mantiene un *recall* superior al 92 % incluso para horizontes > 1 segundo, lo que confirma que la predicción es genuinamente anticipatoria y no una mera detección del cruce en el momento en que ocurre. La Figura 7.3 muestra esta caída suave del *recall* a medida que aumenta el horizonte temporal.

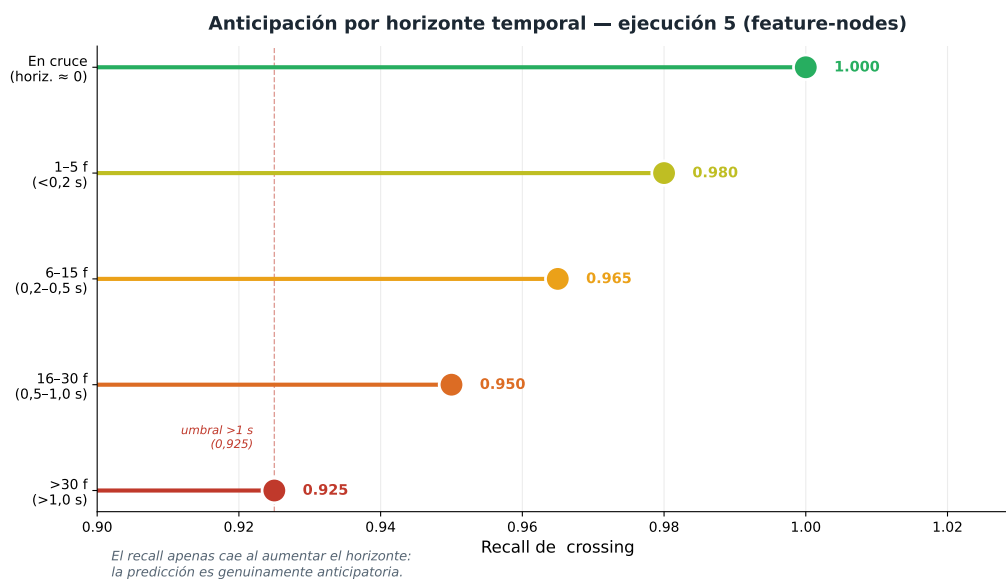


Figura 7.3: Recall de **crossing** por *bucket* de horizonte temporal en la ejecución 5. El color va del verde (horizonte corto) al rojo (horizonte >1 segundo).

7.8 Discusión de la mejora del estilo **feature-nodes**

La mejora de 2,7 pp en AUC-ROC y de 2,9 pp en anticipación del estilo **feature-nodes** frente al estilo **interaction** se explica por el mecanismo de propagación diferenciada: al exponer cada *feature* como un nodo propio, la GNN puede aprender qué señales son más discriminativas para la intención de cruce en cada contexto. Las aristas temporales *feature-a-feature* permiten además que la dinámica de cada señal individual (por ejemplo, la evolución del `tl_time_in_phase` o la trayectoria de `distance`) se propague de forma selectiva, algo que el vector plano de 20 *features* del estilo **interaction** no permite.

Capítulo 8

Discusión y limitaciones

En este capítulo se analiza el significado de los resultados experimentados, compara el trabajo con el contexto más amplio del estado del arte y expone las limitaciones que condicionan su alcance.

8.1 Interpretación de los resultados

El AUC-ROC de 0,764 en el conjunto de prueba indica que el modelo es capaz de ordenar correctamente los peatones que van a cruzar por encima de los que no van a cruzar en el 76,4 % de los pares posibles.

Dado que la evaluación se realiza sobre dos cámaras no vistas en entrenamiento (Walworth Road, Londres; Bourbon Street, Nueva Orleans), el resultado implica que el modelo no ha memorizado las escenas: ha aprendido señales de intención transferibles a entornos nuevos.

La métrica más relevante para el objetivo práctico del sistema —la anticipación temprana— arroja un 92,5 % de *recall* para horizontes superiores a 30 fotogramas (más de 1 segundo antes del cruce a 30 fps). Este valor es superior al obtenido con el estilo *interaction* sobre el mismo split (89,6 %) y comparable al obtenido en la ejecución 2 (92,2 %) que, sin embargo, procedía de un *checkpoint* degenerado con AUC-ROC inferior.

El *recall* de not-crossing de 0,301 indica que el modelo produce una tasa de falsas alarmas relativamente alta: clasifica como crossing a casi el 70 % de los peatones que en realidad no van a cruzar. Esto es coherente con el desbalance residual del dataset (70 % crossing) y con la elección del *pos_weight* automático, que prioriza la anticipación frente a la reducción de falsas alarmas.

Este compromiso entre ambas métricas se aprecia con claridad en la Figura 8.1, que sitúa cada ejecución en función de su *recall* de not-crossing y su anticipación temprana. La ejecución 4, que reduce el *pos_weight*, se desplaza hacia la derecha (menos falsas alarmas) pero cae bruscamente en anticipación; la ejecución 5 permanece en la zona deseable, combinando alta anticipación con el mayor AUC-ROC.

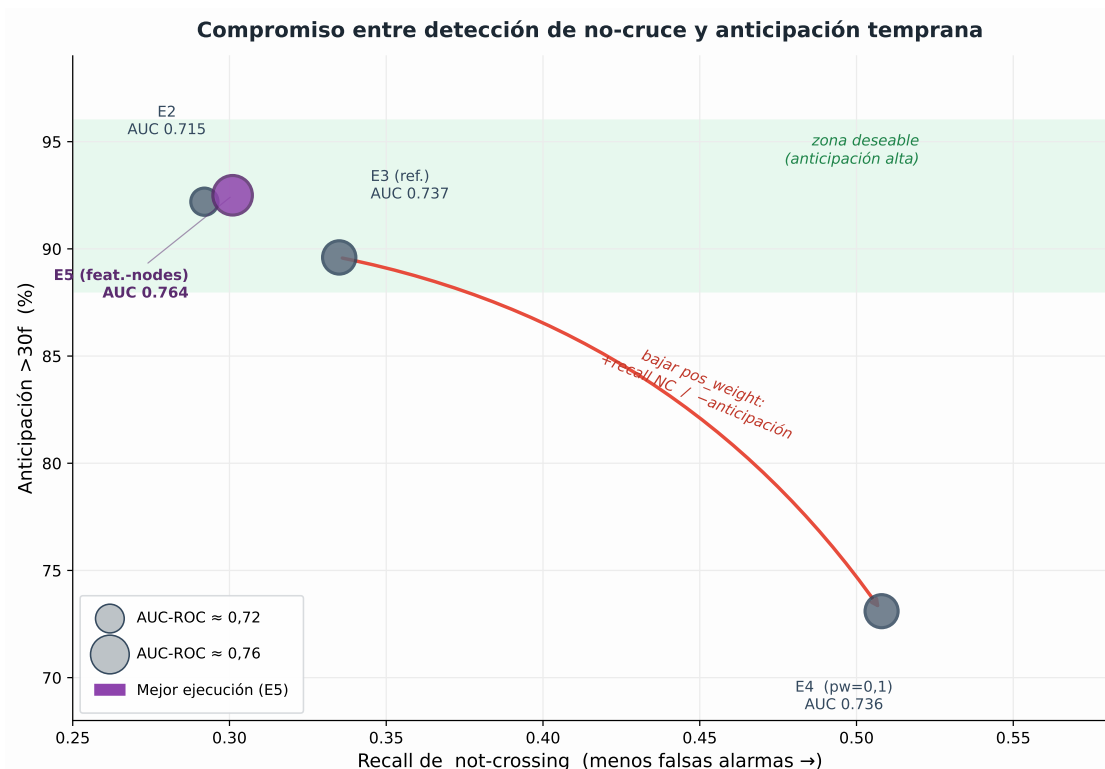


Figura 8.1: Compromiso entre el *recall* de not-crossing y la anticipación temprana. El tamaño de cada burbuja es proporcional al AUC-ROC de la ejecución correspondiente.

8.2 Comparación con el enfoque sobre JAAD (BAL.003)

El trabajo BAL.003 [25] de la misma línea de investigación aplicó una arquitectura GNN espacio-temporal sobre el dataset JAAD, obteniendo resultados competitivos con los modelos de referencia en ese dataset. La comparación directa de cifras de AUC-ROC entre ambos trabajos no es metodológicamente válida, ya que los datasets, los *splits* de test y las distribuciones de clases son distintos.

La aportación diferencial de este trabajo frente al precedente es de naturaleza distinta a la mejora de métrica: mientras que en el trabajo anterior evaluó la arquitectura GNN sobre datos grabados desde vehículo en una única región geográfica, este trabajo demuestra que el mismo enfoque es aplicable a cámaras fijas de infraestructura con cobertura en cinco continentes, y que el *pipeline* de obtención de datos puede construirse de forma completamente automatizada. La contribución es, en este sentido, más de ingeniería de datos y de generalización que de arquitectura de modelo.

8.3 Limitaciones

8.3.1 Dependencia de la calidad del *tracker*

La GNN conecta nodos del mismo peatón entre fotogramas a través del identificador de ByteTrack. Si el *tracker* pierde o reasigna un identificador por oclusión, cruce entre peatones o fondo confuso, la arista temporal correspondiente desaparece y el modelo recibe información incompleta. En escenas con alta densidad de peatones o iluminación deficiente, la tasa de reasignación sube y la calidad de los grafos se degrada de forma silenciosa.

8.3.2 Ventana temporal fija de tres fotogramas

El modelo solo ve tres fotogramas consecutivos. No captura dinámicas que se desarrollan durante más de $\sim 0,1$ segundos (a 30 fps), como la desaceleración progresiva antes de cruzar o patrones de espera de varios segundos. Un contexto más largo (Transformer temporal, RNN) podría mejorar la anticipación en horizontes muy largos, aunque a un coste mayor de parámetros y datos.

8.3.3 Evaluación sobre solo dos cámaras de test

El *split* de test usa el 20 % de las cámaras (2 de 10). No se ha evaluado el modelo sobre cámaras móviles, ángulos picados extremos, resoluciones muy bajas o condiciones nocturnas. Los resultados reportados corresponden a una única partición fija; sin validación cruzada por cámara no es posible estimar la varianza de las métricas.

8.3.4 *Features* estimadas sin calibración métrica

Las variables *distance*, *dist_road_edge* y *cross_horizon* se estiman a partir de la posición del *bounding box* en la imagen 2D, sin información métrica real. No se realiza calibración de cámara ni se dispone de datos de profundidad.

8.3.5 Ausencia de inferencia en tiempo real

El *pipeline* implementado es completamente *offline*: captura $\rightarrow$ etiqueta $\rightarrow$ entrena. No existe un módulo que cargue el *checkpoint* entrenado y clasifique en tiempo real sobre un *stream*.

8.3.6 Balance de clases artificial mediante descarte

criba.py alcanza el 30 % de *not-crossing* descartando secuencias completas, no generando datos nuevos. Esto puede introducir sesgo geográfico si las cámaras más desequilibradas son siempre las mismas. Un *oversampling* a nivel de pista (SMOTE sobre el vector de *features*, o aumentación temporal) sería metodológicamente más limpio.

8.3.7 Sin validación cruzada

Los resultados reportados corresponden a un único *split* train/test. Sin *k-fold leave-one-camera-out* no es posible estimar los intervalos de confianza de las métricas ni confirmar que la elección de las dos cámaras de test no favorece artificialmente los resultados.

Capítulo 9

Conclusiones y trabajo futuro

9.1 Conclusiones

En este trabajo se ha llevado a cabo un sistema completo sobre la intención de cruce de peatones basado en cámaras estáticas públicas. Desde la obtención automatizada de secuencias de videos hasta una posible evaluación de un modelo.

Gracias al enfoque GNN espacio-temporal aplicado a cámaras fijas sobre 8 cámaras de 5 países distintos, se ha obtenido un AUC-ROC de 0,764 y una anticipación temprana del 92,5 % sobre 2 cámaras de test no vistas durante el entrenamiento (Londres y Nueva Orleans). Esto confirma que la representación en grafos espacio-temporales captura señales de intención transferibles entre entornos con infraestructuras, culturas y densidades peatonales distintas.

La calidad del dataset ha sido determinante para los resultados. Las dos primeras ejecuciones del entrenamiento produjeron resultados degenerados (modo colapso, parada temprana prematura) por causas ajenas a la arquitectura propuesta. Algunas de las interrupciones prematuras son: la ausencia de corrección del desbalance de clases y la monitorización incorrecta de la función de parada. A consecuencia de ello, se empezó a tomar la medida de la AUC-ROC y el `pos_weight` automático como medidas de monitorización, aumentando el rendimiento de las métricas con resultados notablemente favorables.

En cuanto al estilo, `feature-nodes` supera al estilo `interaction` en las métricas clave. Representar cada *feature* como un nodo propio en el grafo permite a la GNN aprender qué señales son discriminativas de forma selectiva, mejorando el AUC-ROC en 2,7 pp y la anticipación temprana en 2,9 pp. Esta ganancia se ha conseguido gracias a un coste computacional extra sobre el estilo `interaction`.

Además de lo comentado anteriormente, se ha elaborado un *pipeline* automatizado reproducible y escalable. El sistema de captura y etiquetado puede añadir nuevas cámaras sin modificar el código, solo configurando las máscaras ROI en la primera sesión. El formato de salida es compatible con el dataset JAAD, lo que facilita la comparación con trabajos previos y la extensión del sistema según necesidades geográficas o de otra índole.

Finalmente, se ha conseguido un 92,5% de *recall* para horizontes superiores a 1 segundo antes del cruce, indicando que el sistema podría aportar un margen de reacción útil en aplicaciones de seguridad vial, aunque con la salvedad importante del *recall* de `not-crossing` de 0,301, que implica una tasa de falsas alarmas a evaluar en contexto de despliegue real.

9.2 Trabajo futuro

Para discernir sobre las posibles mejoras o evoluciones del trabajo se han considerado tres divisiones temporales: corto, medio y largo plazo. Todo ello para que se pueda enfocar según las necesidades presentes.

9.2.1 Corto plazo

Inferencia en tiempo real. El paso más inmediato es implementar un módulo que cargue el *checkpoint* entrenado y ejecute el *pipeline*: detección $\rightarrow$ construcción de grafo $\rightarrow$ predicción sobre cada fotograma de un *stream* en vivo, con *overlay* de la predicción sobre el vídeo. Los bloques individuales ya existen; falta integrarlos en un bucle de baja latencia.

Validación cruzada por cámara: aplicar *k-fold leave-one-camera-out* para obtener intervalos de confianza reales sobre AUC-ROC y anticipación, y confirmar que los resultados reportados no dependen de la elección particular de las dos cámaras de test.

Ampliación del dataset. El *pipeline* está diseñado para escalar sin intervención manual relevante. Añadir más cámaras de regiones no representadas (África, América del Sur, Oriente Medio) aumentaría la diversidad y la robustez del modelo.

9.2.2 Medio plazo

Contexto temporal más largo. Sustituir el triplete de fotogramas por una ventana deslizante de 8–16 fotogramas con aristas recurrentes, o aplicar un Transformer temporal sobre los *embeddings* de nodo. Con finalidad para capturar dinámicas de mayor duración como la desaceleración progresiva previa al cruce.

Calibración de cámara. Usar un patrón de calibración o puntos de referencia conocidos en la escena para obtener estimaciones métricas reales de distancia y velocidad, mejorando la fiabilidad de las *features* distance, dist_road_edge y cross_horizon, entre otras.

Integración del semáforo como nodo de grafo. Actualmente el semáforo se incluye como atributo del nodo peatón. Modelarlo como un nodo propio en el grafo, conectado a todos los peatones del fotograma, permitiría que la GNN propagara su influencia de forma más explícita.

9.2.3 Largo plazo

Modelo ligero para despliegue en infraestructura. Destilar el modelo GNN en una versión compacta (*pruning* + cuantización) ejecutable en dispositivos de bajo consumo (Raspberry Pi, NVIDIA Jetson Nano) para su instalación directa en semáforos o cámaras de intersecciones conflictivas.

Dataset abierto. Publicar `etiquetas_ALL.csv` como dataset público de intención de cruce en cámara fija, complementario al dataset JAAD. Constituiría el primer dataset de este tipo con cobertura geográfica en cinco continentes.

Fusión multi-cámara. Combinar grafos de varias cámaras que cubran el mismo cruce para obtener una representación 3D implícita del comportamiento peatonal, superando las limitaciones de la perspectiva 2D de las cámaras individuales.

Bibliografía

- [1] World Health Organization, “Global Status Report on Road Safety 2023”, World Health Organization, Geneva, inf. téc., 2023. dirección: <https://www.who.int/publications/i/item/9789240086517>.
- [2] A. Rasouli, I. Kotseruba y J. K. Tsotsos, “Are They Going to Cross? A Benchmark Dataset and Baseline for Pedestrian Crosswalk Behavior”, en *Proceedings of the IEEE International Conference on Computer Vision Workshops (ICCVW)*, 2017, págs. 206-213. DOI: [10.1109/ICCVW.2017.33](https://doi.org/10.1109/ICCVW.2017.33).
- [3] A. Rasouli, I. Kotseruba, T. Kunic y J. K. Tsotsos, “PIE: A Large-Scale Dataset and Models for Pedestrian Intention Estimation and Trajectory Prediction”, en *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, págs. 6262-6271. DOI: [10.1109/ICCV.2019.00636](https://doi.org/10.1109/ICCV.2019.00636).
- [4] A. N. Melo Castillo, “Explainable Features and Models for Road User’s Behavior Understanding and Prediction”, Ph.D. Thesis, Ph.D. Program in Information and Communications Technologies, Universidad de Alcalá, 2025.
- [5] C. Morris et al., “Weisfeiler and Leman Go Neural: Higher-Order Graph Neural Networks”, en *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, págs. 4602-4609. DOI: [10.1609/aaai.v33i01.33014602](https://doi.org/10.1609/aaai.v33i01.33014602).
- [6] H. Sang, W. Chen y Z. Zhao, “Review of pedestrian trajectory prediction based on graph neural networks”, *Information Fusion*, 2025, Available online 11 September 2025. dirección: <https://www.sciencedirect.com/science/article/pii/S1566253525007894>.
- [7] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library”, en *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019. dirección: <https://arxiv.org/abs/1912.01703>.
- [8] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang y P. S. Yu, “A Comprehensive Study of Graph Neural Networks”, *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, n° 1, págs. 4-24, 2021. DOI: [10.1109/TNNLS.2020.2978386](https://doi.org/10.1109/TNNLS.2020.2978386).
- [9] T. N. Kipf y M. Welling, “Semi-Supervised Classification with Graph Convolutional Networks”, en *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017. dirección: <https://arxiv.org/abs/1609.02907>.
- [10] B. Yu, H. Yin y Z. Zhu, “Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting”, en *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence (IJCAI)*, 2018, págs. 3634-3640. DOI: [10.24963/ijcai.2018/505](https://doi.org/10.24963/ijcai.2018/505).
- [11] Ultralytics, *YOLO26 by Ultralytics*, <https://github.com/ultralytics/ultralytics>, Accessed 2025, 2025.

- [12] G. Qin, R. Pan, Y. Deng, P. Mi e Y. Zhu, “Road pedestrian detection and tracking algorithm based on improved YOLOv5s and DeepSORT”, *PLOS ONE*, vol. 20, n° 11, e0334786, 2025. DOI: [10.1371/journal.pone.0334786](https://doi.org/10.1371/journal.pone.0334786).
- [13] F. Zhang, L. V. Leong, K. S. Yen e Y. Zhang, “An enhanced lightweight model for small-scale pedestrian detection based on YOLOv8s”, *Digital Signal Processing*, 2024. DOI: [10.1016/j.dsp.2024.104866](https://doi.org/10.1016/j.dsp.2024.104866).
- [14] W. Farhat, O. Ben Rhaïem, H. Faïedh y C. Souani, “Pedestrian detection and tracking using an enhanced YOLOv9 model for automotive vehicles”, *Measurement*, 2025. DOI: [10.1016/j.measurement.2025.118009](https://doi.org/10.1016/j.measurement.2025.118009).
- [15] Z. Qu et al., “LP-YOLO: An improved lightweight pedestrian detection algorithm based on YOLOv11”, *Digital Signal Processing*, vol. 165, pág. 105343, 2025. DOI: [10.1016/j.dsp.2025.105343](https://doi.org/10.1016/j.dsp.2025.105343).
- [16] Q. Liu, Z. Li, L. Zhang y J. Deng, “MSCD-YOLO: A Lightweight Dense Pedestrian Detection Model with Finer-Grained Feature Information Interaction”, *Sensors*, vol. 25, pág. 438, 2025. DOI: [10.3390/s25020438](https://doi.org/10.3390/s25020438).
- [17] S. Huang, S. Zhang e Y. Jiao, “Ms-VLPD: A multi-scale VLPD based method for pedestrian detection”, *Expert Systems With Applications*, vol. 271, pág. 126613, 2025. DOI: [10.1016/j.eswa.2025.126613](https://doi.org/10.1016/j.eswa.2025.126613).
- [18] W. Liu, X. Qiao, C. Zhao, T. Deng y F. Yan, “VP-YOLO: A human visual perception-inspired robust vehicle-pedestrian detection model for complex traffic scenarios”, *Expert Systems With Applications*, 2025. DOI: [10.1016/j.eswa.2025.126837](https://doi.org/10.1016/j.eswa.2025.126837).
- [19] Z. Chen, Y. Qian, X. Yang, C. Wang y M. Yang, “AMFD: Distillation via Adaptive Multimodal Fusion for Multispectral Pedestrian Detection”, *IEEE Transactions on Multimedia*, vol. 27, págs. 8298-8311, 2025. DOI: [10.1109/TMM.2025.3604937](https://doi.org/10.1109/TMM.2025.3604937).
- [20] H. Wang et al., “RGB-FIR Multimodal Pedestrian Detection with Cross-Modality Context Attentional Model”, *Sensors*, vol. 25, pág. 3854, 2025. DOI: [10.3390/s25133854](https://doi.org/10.3390/s25133854).
- [21] R. Zhao et al., *PedDet: Adaptive Spectral Optimization for Multimodal Pedestrian Detection*, arXiv preprint arXiv:2502.14063, 2025.
- [22] Y. Zhang et al., “ByteTrack: Multi-Object Tracking by Associating Every Detection Box”, en *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022, págs. 1-21. DOI: [10.1007/978-3-031-20047-2_1](https://doi.org/10.1007/978-3-031-20047-2_1).
- [23] M. Fey y J. E. Lenssen, “Fast Graph Representation Learning with PyTorch Geometric”, en *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019. dirección: <https://arxiv.org/abs/1903.02428>.
- [24] W. Falcon y The PyTorch Lightning Team, *PyTorch Lightning*, <https://github.com/Lightning-AI/lightning>, Version 2.6.1, accessed 2024, 2023.
- [25] A. Rodríguez Hurtado, “Estimación de la intención de los usuarios de la vía mediante redes neuronales basadas en grafos”, Trabajo de Fin de Grado, Universidad de Alcalá, 2026.



Universidad
de Alcalá

2026